



Lecture 5: Language Model

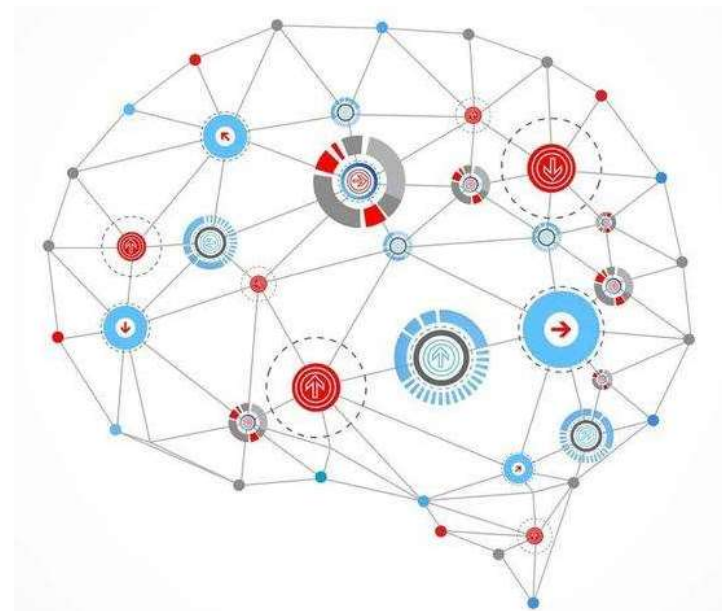


Xu Ruifeng

Harbin Institute of Technology, Shenzhen

Last Time

- From ML to DL
- Neuron and ANN
- DL models
 - RNN 循环神经网络
 - CNN 卷积神经网络
 - Attention 注意力机制





Today's class

- **Language Modeling**
- Statistical Language Model
 - N-gram
 - Smoothing 平滑
- Neural Language Model
 - NNLM
 - Pre-trained Language Model

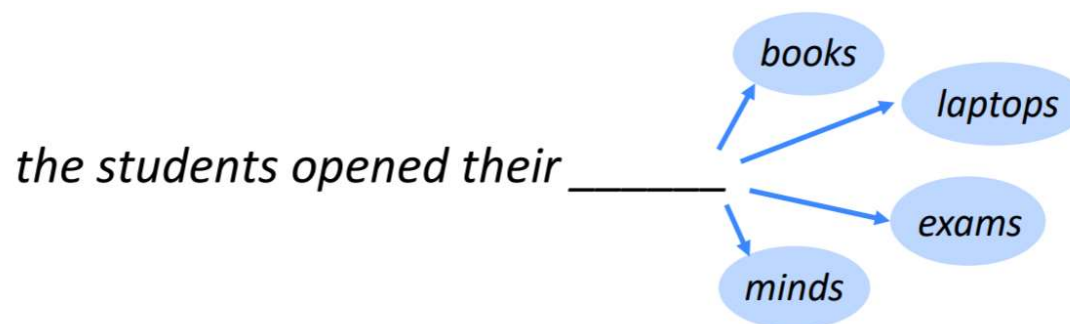


Language Modeling



Language Modeling

- Language Modeling
 - To predict what word comes next.



- Formally, given a sequence of words $w_1, w_2, \dots, w_t$, compute the probability distribution of the next words w_{t+1} .

$$p(w_{t+1} | w_t, \dots, w_1)$$



Language Modeling





Language Modeling



语言模型的



百度一下

语言模型的功能

语言模型的应用

语言模型的参数估计经常使用MLE

语言模型的类型

语言模型的损失函数

语言模型的概念

语言模型的缺点

语言模型的训练

反馈

图 采购 更多

搜索工具

NLP)的基础模型,是

odel)即是用来描述
分布的模型,能够用



Language Modeling

- Language Modeling

- Further, we can assign a probability to an entire sentence.

all of a sudden I notice three guys standing on the sidewalk ✓

on guys all I of notice sidewalk three a sudden standing the ✗

- Decompose this probability by the chain rule of probability.

$$p(w_1, w_2, \dots, w_T) = p(w_1)p(w_2|w_1)p(w_3|w_2, w_1) \dots p(w_T|w_{T-1}, \dots, w_1)$$



Language Modeling

- Language Modeling

all of a sudden I notice three guys standing on the sidewalk ✓

on guys all I of notice sidewalk three a sudden standing the ✗

- Models that assign probabilities to sequences of words are called
Language Models or LMs.

Language Modeling

- Application: Speech Recognition



- I went to a party. / Eye went two a bar tea.
- 你现在在干什么 / 你西安载感什么
- 我劝你好自为之 / 我劝你耗子尾汁



Language Modeling

- Application: Machine Translation

他 向 记者 介绍了 主要 内容
He to reporters introduced main content

- Potential rough English translations

he introduced reporters to the main contents of the statement

he briefed to reporters the main contents of the statement

he briefed reporters on the main contents of the statement

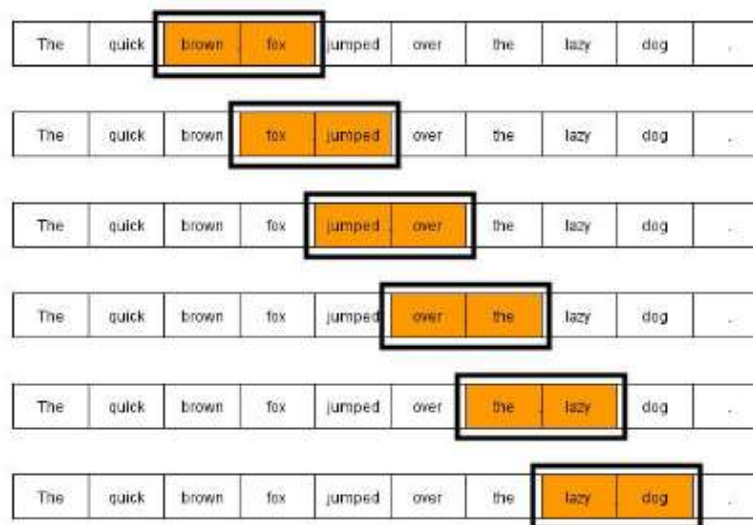


Language Modeling

- Other Applications
 - OCR & Handwriting recognition
 - More probable sentences are more likely correct readings.
 - Context sensitive spelling correction
 - 马老师，花生神么事情了
 - *Their are problems wit this sentence*
 - Chinese word segmentation
 - 这 / 篇 / 文章 / 写 / 得 / 太 / 平淡 / 了 / 。
 - 这 / 篇 / 文章 / 写 / 得 / 太平 / 淡 / 了 / 。



- Language Modeling
- **Statistical Language Model**
 - N-gram
 - Smoothing 平滑
- Neural Language Model
 - NNLM
 - Pre-trained Language Model





N-gram

- Motivation
 - Estimate the probability $p(w|h)$ of a word w given some history h
 - E.g. history: “*its water is so transparent that*”, next word “*the*”
 - count and divide

$$P(\text{the}|\text{its water is so transparent that}) = \frac{C(\text{its water is so transparent that the})}{C(\text{its water is so transparent that})}$$

- No corpus is **big enough** to give you good estimates



N-gram

- Simplifying Assumption

- Instead of computing the probability of a word given its entire history, we can **approximate the history** by just the last few words.



Andrei Markov

- Markov assumption
 - the probability of a word depends only on the previous word
- Bigram model

$$P(\text{the}|\text{its water is so transparent that}) \approx P(\text{the}|\text{that})$$



N-gram

- Bigram, Trigram, N-gram
 - We can generalize the bigram to
 - the **trigram**: looks two words into the past
 - the **n-gram**: looks $n-1$ words into the past



N-gram

- Bigram
 - Estimating bigram probabilities: count and divide

$$P(w_t | w_{t-1}, w_{t-2}, \dots, w_1) \approx P(w_t | w_{t-1}) = \frac{C(w_{t-1} w_t)}{C(w_{t-1})}$$

- The probabilities to sequences of words
 - to have a consistent probabilistic model, append a unique start and end symbol to every sentence and treat these as additional words.

$$P(<S>, w_1, w_2, \dots, w_T, </S>) \approx P(w_1 | <S>) P(w_2 | w_1) \dots P(w_T | w_{T-1}) P(</S> | w_T)$$



Example for Bigram

- Example1
 - given a corpus of three sentences

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

- bigram probabilities

$$P(I | <s>) = \frac{2}{3} = .67 \quad P(\text{Sam} | <s>) = \frac{1}{3} = .33 \quad P(\text{am} | I) = \frac{2}{3} = .67$$

$$P(</s> | \text{Sam}) = \frac{1}{2} = 0.5 \quad P(\text{Sam} | \text{am}) = \frac{1}{2} = .5 \quad P(\text{do} | I) = \frac{1}{3} = .33$$



Example for Bigram

- Example2: Berkeley Restaurant Project corpus of 9332 sentences
 - Unigram count

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

- Bigram count

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0



Example for Bigram

- Example2: Berkeley Restaurant Project corpus of 9332 sentences
 - Bigram probabilities

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

$$P(\text{want}|\text{i}) = \frac{C(\text{i want})}{C(\text{i})} = \frac{827}{2533} = 0.33$$



Example for Bigram

- Example2: Berkeley Restaurant Project corpus of 9332 sentences
 - Bigram probabilities

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

$$\begin{aligned} & P(< s > i \text{ want chinese food} < / s >) \\ &= P(i | < s >) P(\text{want} | i) P(\text{chinese} | \text{want}) P(\text{food} | \text{chinese}) P(< / s > | \text{food}) \\ &= 0.25 \times 0.33 \times 0.0065 \times 0.52 \times 0.68 \\ &= 0.00019 \end{aligned}$$



Example for Bigram

- Example2: Berkeley Restaurant Project corpus of 9332 sentences
 - Bigram probabilities

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

- $P(i \text{ want to eat chinese food})?$



Train and Test Corpora

- A language model must be trained on a large corpus of text to **estimate good parameter values**.
- Model can be evaluated based on its ability to predict a high probability for a disjoint (held-out) test corpus (testing on the training corpus would give an optimistically biased estimate).



Unknown Words

- How to handle words in the test corpus that did not occur in the training data, i.e. **out of vocabulary** (OOV) words?
- Train a model that includes an explicit symbol for an unknown word (<UNK>).
 - Choose a vocabulary in advance and replace other words in the training corpus with <UNK>.
 - Replace the first occurrence of each word in the training data with <UNK>.



Generalization

- Word sequences

$$w_1^n = w_1 \dots w_n$$

- Chain rule of probability

$$P(w_1^n) = P(w_1)P(w_2 | w_1)P(w_3 | w_1^2) \dots P(w_n | w_1^{n-1}) = \prod_{k=1}^n P(w_k | w_1^{k-1})$$

- N-gram approximation

$$P(w_1^n) = \prod_{k=1}^n P(w_k | w_{k-N+1}^{k-1})$$



Generalization

- N-grams do a better and better job of modeling the training corpus as we increase the value of N.

1
gram

–To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have

–Hill he late speaks; or! a more to leg less first you enter

2
gram

–Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow.

–What means, sir. I confess she? then all sorts, he is trim, captain.

3
gram

–Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done.

–This shall forbid it should be branded, if renown made it empty.

4
gram

–King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in;

–It cannot be but so.



Problem for N-Grams: Long Distance Dependencies

- Many times local context does not provide the most useful predictive clues, which instead are provided by ***long-distance dependencies***.
 - Syntactic dependencies
 - “The **man** next to the large oak tree near the grocery store on the corner **is** tall.”
 - “The **men** next to the large oak tree near the grocery store on the corner **are** tall.”
 - Semantic dependencies
 - “The **bird** next to the large oak tree near the grocery store on the corner **flies** rapidly.”
 - “The **man** next to the large oak tree near the grocery store on the corner **talks** rapidly.”
 - Modifiers lead to different distances
- More complex models of language are needed to handle such dependencies.



Why use only bi- or tri-grams?

- Markov approximation is still costly
 - with a 20 000 word vocabulary:
 - Bigram needs to store 400 million parameters
 - Trigram needs to store 8 trillion parameters
 - Using a language model > trigram is impractical
- To reduce the number of parameters, we can:
 - Do stemming (use stems instead of word types)
 - Group words into semantic classes
 - Seen once --> same as unseen



Distant Bi-gram

- Apply bi-gram with distance to estimate N-gram

$$P(w_1^n) = P(w_1)P(w_2 | w_1)P(w_3 | w_1^2) \dots P(w_n | w_1^{n-1}) = \prod_{k=1}^n P(w_k | w_{k-d}, d)$$

$k=1$
 $d=1, \text{window size}$

- Linearly increased parameter space
- Better than Bi-gram
- A little better than Tri-gram
- But lower than N-Gram



Other Language Modeling Techniques

- Caching
 - Based on the observation that recently observed words are likely to occur again. Models from recently observed data can be combined with more general models to improve performance.
 - If you say something, you are likely to say it again later.
 - Interpolate trigram with cache.

$$P(z \mid history) \approx \lambda P_{smooth}(z \mid xy) + (1 - \lambda) P_{cache}(z \mid history)$$

$$P_{cache}(z \mid history) = \frac{C(z \in history)}{length(history)}$$



Evaluation of Language Models

- The Shannon Game

- how well can we predict the next word?

I always order pizza with cheese and _____

mushrooms 0.1
pepperoni 0.1
anchovies 0.01
....
fried rice 0.0001
....
and 1e-100

- A better model of a text

- is one which assigns a higher probability to the word that actually occurs



Evaluation of Language Models

- Perplexity 困惑度

- Measure of how well a model “fits” the test data
- For a sentence $S = w_1 w_2 \cdots w_T$

$$\begin{aligned} PP(S) &= P(w_1 w_2 \dots w_N)^{-\frac{1}{N}} \\ &= \sqrt[N]{\frac{1}{p(w_1 w_2 \dots w_N)}} \\ &= \sqrt[N]{\prod_{i=1}^N \frac{1}{p(w_i | w_1 w_2 \dots w_{i-1})}} \end{aligned}$$

- Minimizing Perplexity is the same as maximizing probability



Evaluation of Language Models

- Perplexity: lower perplexity = better model
 - LM on 38 million words from the Wall Street Journal (WSJ) using a 19,979 word vocabulary

	Unigram	Bigram	Trigram
Perplexity	962	170	109

- The more information the n-gram gives us about the word sequence, the lower the perplexity



Smoothing

- Zeros
 - Many rare (but not impossible) combinations never occur in training, **zero** will be assigned.

- | | |
|--|--|
| <ul style="list-style-type: none">• Training set:<ul style="list-style-type: none">... denied the allegations... denied the reports... denied the claims... denied the request <p>$P(\text{"offer"} \mid \text{denied the}) = 0$</p> | <ul style="list-style-type: none">• Test set<ul style="list-style-type: none">... denied the offer... denied the loan |
|--|--|

- If a new combination occurs during testing, it is given a probability of zero and **the entire sequence gets a probability of zero**.
- e.g. $P(w_t | w_{t-1}) = 0 \Rightarrow P(S) = 0$



Smoothing

- Smoothing

- When we have sparse statistics:

$P(w \mid \text{denied the})$

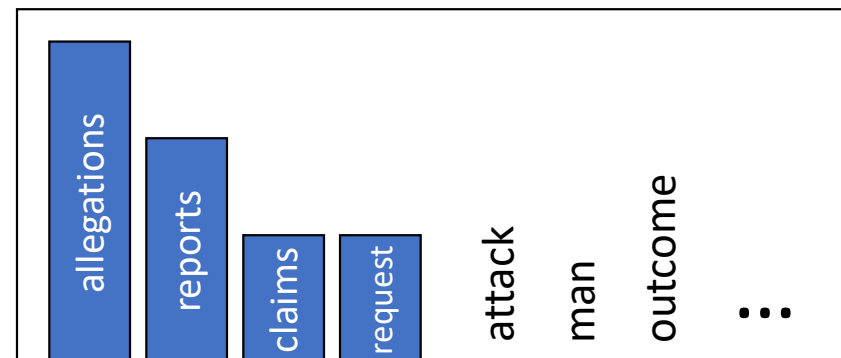
3 allegations

2 reports

1 claims

1 request

7 total



- Steal probability mass to generalize better

$P(w \mid \text{denied the})$

2.5 allegations

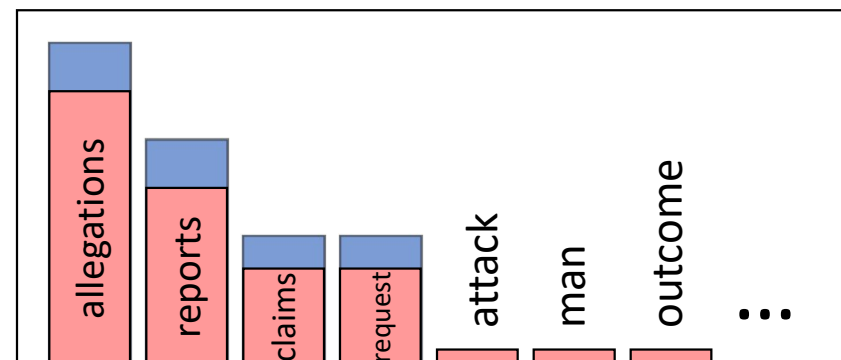
1.5 reports

0.5 claims

0.5 request

2 other

7 total





Smoothing

- Smoothing
 - we need to allow for possibility of seeing events not seen in training
 - decrease the probability of previously seen events to leave a little bit of probability for previously unseen events





Laplace (Add-One) Smoothing

- Pretend we saw each word one more time than we did
- Just add one to all the counts


$$P(w_t|w_{t-1}) = \frac{C(w_t w_{t-1})}{C(w_t)}$$
$$P(w_t|w_{t-1}) = \frac{C(w_t w_{t-1}) + 1}{C(w_t) + V}$$

- What happens to our P values if we don't increase the denominator?



Laplace (Add-One) Smoothing

- N-gram


$$P(w_t | w_{t-1} \cdots w_{t-n-1}) = \frac{C(w_t w_{t-1} \cdots w_{t-n-1})}{C(w_t)}$$
$$P(w_t | w_{t-1} \cdots w_{t-n-1}) = \frac{C(w_t w_{t-1} \cdots w_{t-n-1}) + 1}{C(w_t) + V^{n-1}}$$



Laplace (Add-One) Smoothing

- Discount: the ratio of the discounted counts to the original counts:

$$c_i^* = (c_i + 1) \frac{N}{N + V}$$

$$d_c = \frac{c^*}{c}$$



Laplace (Add-One) Smoothing

- Bigram counts

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0



Laplace (Add-One) Smoothing

- Laplace smoothed bigram counts

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1



Laplace (Add-One) Smoothing

- Laplace smoothed bigram probabilities

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00078	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058



Laplace (Add-One) Smoothing

- Reconstituted counts

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16

$$c^*(w_{n-1}w_n) = \frac{[C(w_{n-1}w_n) + 1] \times C(w_{n-1})}{C(w_{n-1}) + V}$$



Laplace (Add-One) Smoothing

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16



Laplace (Add-One) Smoothing

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16





Add-K Smoothing

- Instead of adding 1 to each count, we add a fractional count k (.5? .05? .01?).

$$P(w_t | w_{t-1}) = \frac{C(w_t w_{t-1}) + k}{C(w_t) + kV}$$



Advanced Smoothing

- Many advanced techniques have been developed to improve smoothing for language models.
 - Good-Turing estimate
 - Back-off
 - Interpolation
 - Kneser-Ney Smoothing



Back-off and Interpolation

- Back-off 退避
 - use trigram if you have good evidence,
 - otherwise bigram, otherwise unigram
- Interpolation 插值
 - mix unigram, bigram, trigram
- Interpolation works better



Back-off

- Only use lower-order model when data for higher-order model is unavailable (i.e. count is zero).
- Recursively back-off to weaker models until data is available.

$$P_{katz}(w_n | w_{n-N+1}^{n-1}) = \begin{cases} P^*(w_n | w_{n-N+1}^{n-1}) & \text{if } C(w_{n-N+1}^{n-1}) > 1 \\ \alpha(w_{n-N+1}^{n-1}) P_{katz}(w_n | w_{n-N+2}^{n-1}) & \text{otherwise} \end{cases}$$

- Where P^* is a discounted probability estimate to reserve mass for unseen events and α 's are back-off weights



Linear Interpolation

- Simple interpolation

$$\begin{aligned}\hat{P}(w_t|w_{t-1}w_{t-2}) = & \lambda_1 P(w_t|w_{t-1}w_{t-2}) \\ & + \lambda_2 P(w_t|w_{t-1}) \\ & + \lambda_3 P(w_t)\end{aligned}\quad \sum_i \lambda_i = 1$$

- Lambdas conditional on context

$$\begin{aligned}\hat{P}(w_t|w_{t-1}w_{t-2}) = & \lambda_1(w_{t-1}w_{t-2})P(w_t|w_{t-1}w_{t-2}) \\ & + \lambda_2(w_{t-1}w_{t-2})P(w_t|w_{t-1}) \\ & + \lambda_3(w_{t-1}w_{t-2})P(w_t)\end{aligned}$$



Linear Interpolation

- Use a **held-out** corpus

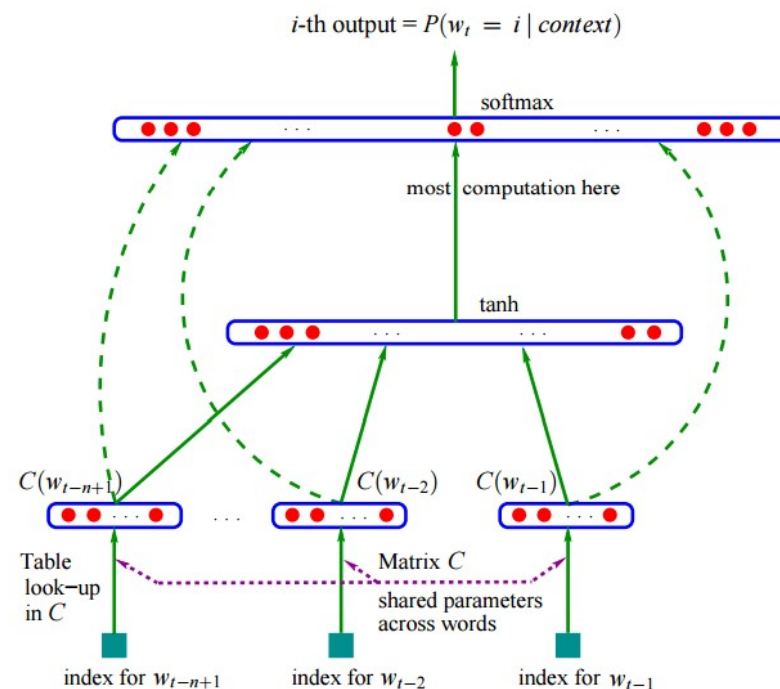


- Choose λ s to maximize the probability of held-out data
 - fix the N-gram probabilities (on the training data).
 - then search for λ s that give largest probability to held-out set.



Today's class

- Language Modeling
- Statistical Language Model
 - N-gram
 - Smoothing 平滑
- **Neural Language Model**
 - NNLM
 - Pre-trained Language Model





Weakness of SLM

- The weakness of statistical language model

该 课程 很 枯燥 ， 大家 觉得 很 无聊 。

- sparse data
 - “很 无聊” never occurs in training set
- semantic similarity is ignored
 - although “无聊” and “枯燥” are semantically similar, they cannot share information



Word Representation

- One-hot Representation

w_0 =该 w_1 =课程 w_2 =很 w_3 =枯燥 w_4 =,
 w_5 =大家 w_6 =觉得 w_7 =很 w_8 =无聊 w_9 =。

$|V|$

$\begin{bmatrix} \vdots \end{bmatrix}$



所有词按照出现的顺序排序



每个词语将对应唯一的下标

枯燥

$\begin{bmatrix} 0 \\ \mathbf{1} \\ 0 \\ \vdots \\ 0 \\ 0 \\ 0 \end{bmatrix}$

无聊

$\begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \\ \mathbf{1} \\ 0 \end{bmatrix}$

High-dimensional, discrete and sparse



Word Representation

- One-hot Representation

$$\begin{array}{cc} \text{枯燥} & \text{无聊} \\ \begin{bmatrix} 0 \\ \color{red}{1} \\ 0 \\ \vdots \\ 0 \\ 0 \\ 0 \end{bmatrix} & \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \\ \color{red}{1} \\ 0 \end{bmatrix} \end{array}$$

枯燥 $\otimes$ 无聊

$$\begin{bmatrix} 0 \\ \color{red}{1} \\ 0 \\ \vdots \\ 0 \\ 0 \\ 0 \end{bmatrix} \times \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \\ \color{red}{1} \\ 0 \end{bmatrix}$$

$= 0$

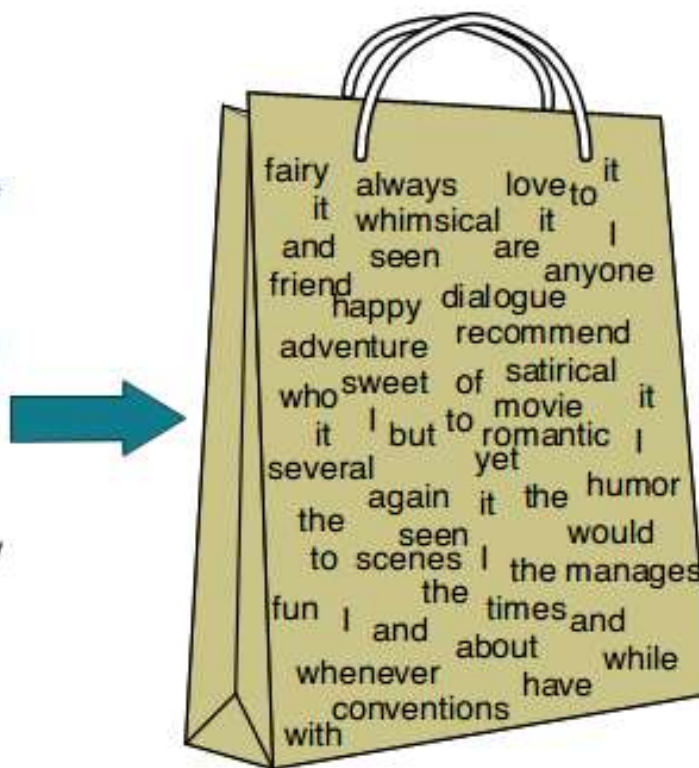


The similarity between any two words is 0.

Bag-of-words

- Bag-of-words

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
...	...



TF-IDF

- There are some problems using 0/1 or word frequency to represent words
 - The use of 0/1 only emphasizes the difference between the words that appear and the words that do not appear (it may perform well in some short text classifications), but does not reflect the difference between the words that appear.
 - When used frequently, some common words appear in many articles, such as "I, is, because". These high-frequency words do not contribute much to the topic of the document. On the contrary, some low-frequency words, such as "bees, farming, machine tools", can better reflect the theme of a document.
- Therefore, assigning weights to term frequency (TF), that is, inverse document frequency (IDF), to obtain TF-IDF values. The higher the importance of a word to an article, the greater its TF-IDF value.



TF-IDF

- Term Frequency (TF)

$$TF_{w,D_i} = \frac{\text{count}(w)}{|D_i|}$$

- Inverse Document Frequency (IDF)

$$IDF_w = \log \frac{N}{\sum_{i=1}^N I(w, D_i)}$$

- To avoid a denominator of 0, smooth the IDF

$$IDF_w = \log \frac{N}{1 + \sum_{i=1}^N I(w, D_i)}$$

- TF-IDF value can be obtained by multiplying TF and IDF

$$TF - IDF_{w,D_i} = TF_{w,D_i} * IDF_w$$

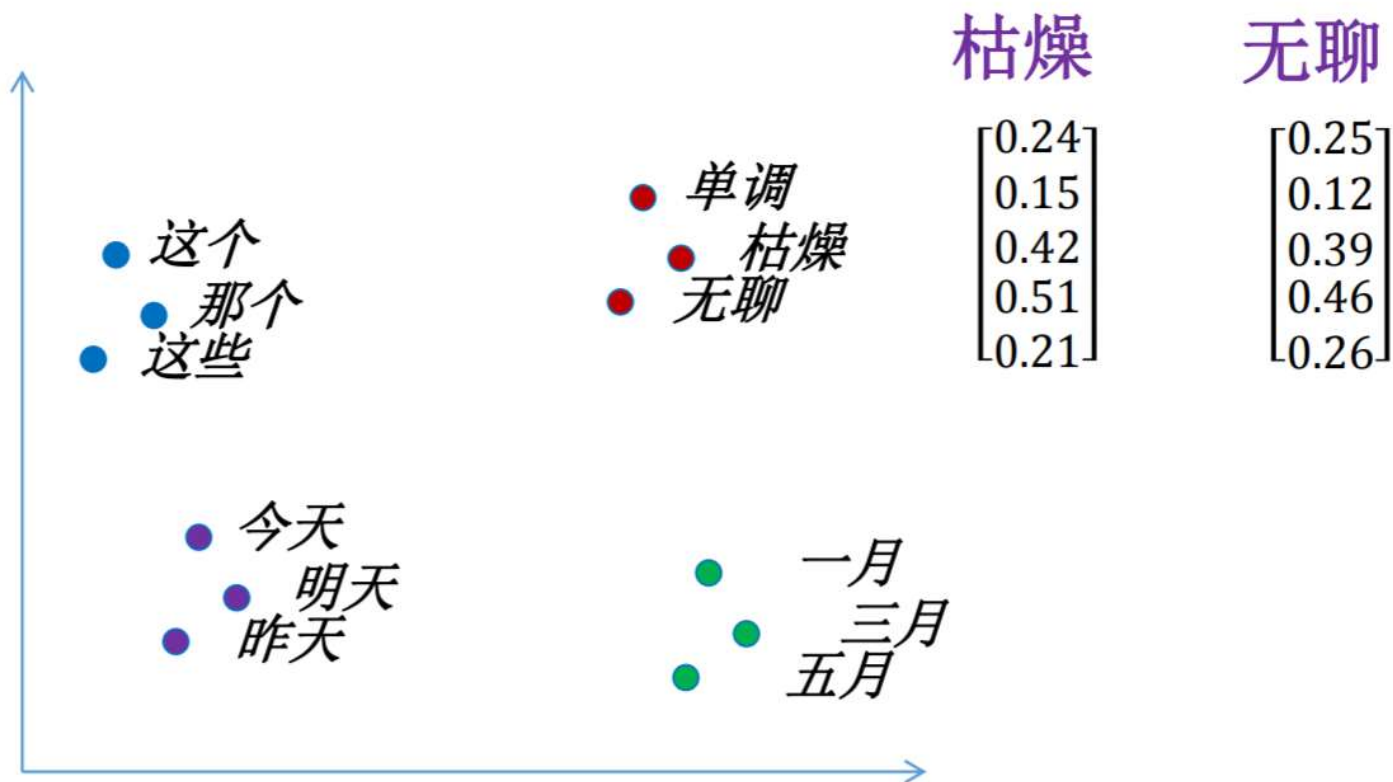


Real world VS. Cognitive world

- Real world: objects exist independently of each other
- Cognitive world: concepts are interconnected and semantically distributed continuously



Word representation on continuous semantic space



Low-dimensional, dense continuous real number space



Word Embedding

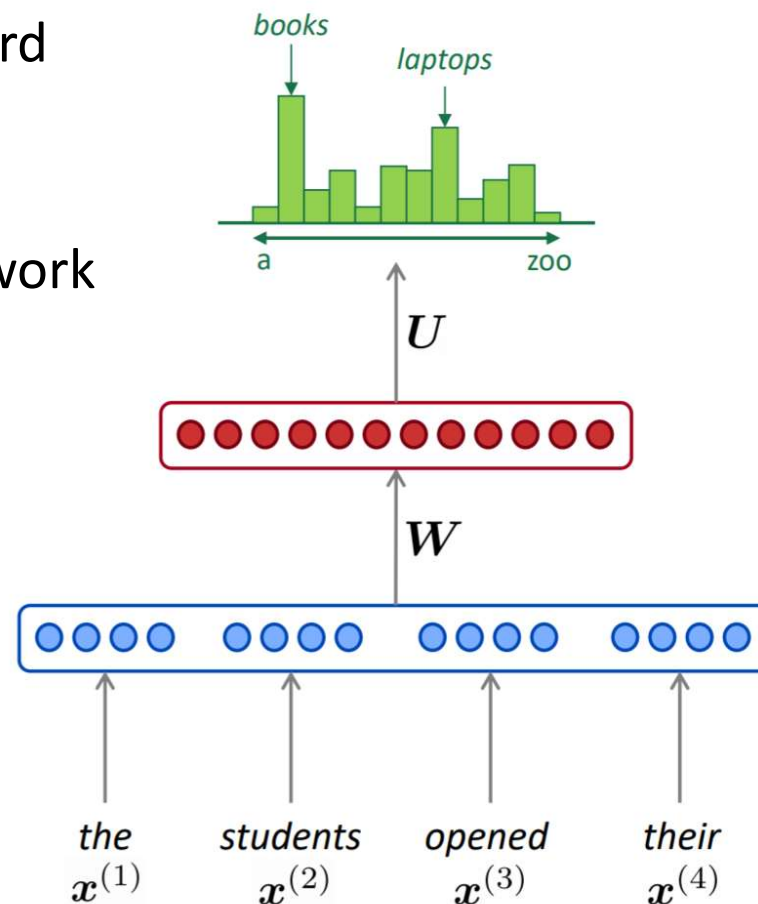
$$L = \begin{bmatrix} \text{枯燥} & \dots & \text{单调} & \text{无聊} \\ \vdots & & \vdots & \vdots \\ \vdots & & \vdots & \vdots \\ \vdots & & \vdots & \vdots \\ \vdots & & \vdots & \vdots \end{bmatrix} \quad \begin{matrix} V \\ D \end{matrix} \quad L \in \mathbb{R}^{D \times V}$$

Diagram illustrating a word embedding matrix L . The matrix is shown as a grid of red dots, with columns representing words (枯燥, ..., 单调, 无聊) and rows representing dimensions. The first column is highlighted with a purple box. The matrix is labeled $L \in \mathbb{R}^{D \times V}$, where D is the dimensionality and V is the vocabulary size.



NNLM

- Neural Network Language Model
 - suppose we already have a learned word embedding matrix
 - let's predict next words by Neural Network





NNLM

- Neural Network Language Model

- hidden layer

$$x = (C(w_{t-1}), C(w_{t-2}), \dots, C(w_{t-n+1})).$$

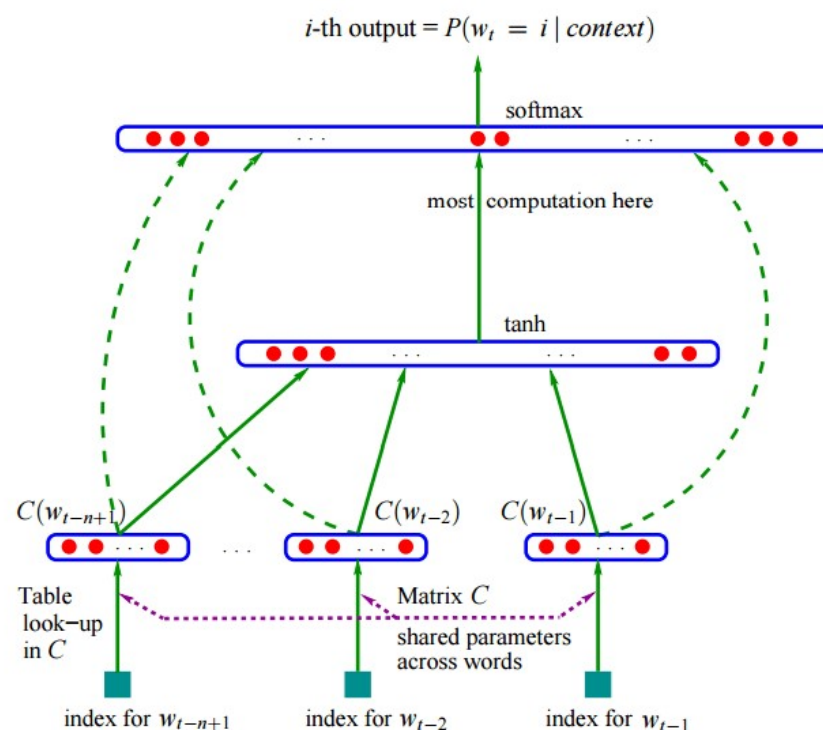
$$h = \tanh(d + Hx)$$

where $x \in R^{n \times D}$, $y \in R^V$

- output layer

$$y = b + Uh$$

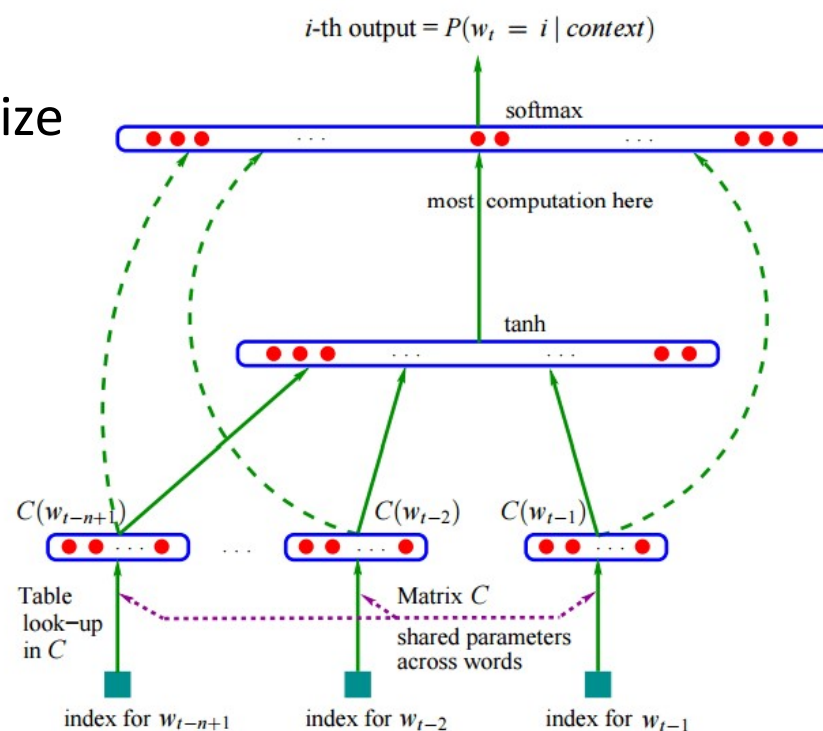
$$\hat{P}(w_t | w_{t-1} \dots w_{t-n-1}) = \text{softmax}(y_{w_t})$$





NNLM

- The word vector appears in two places
 - $C \in R^{D \times V}$
 - $U \in R^{|h| \times V}$, where $|h|$ is the hidden size



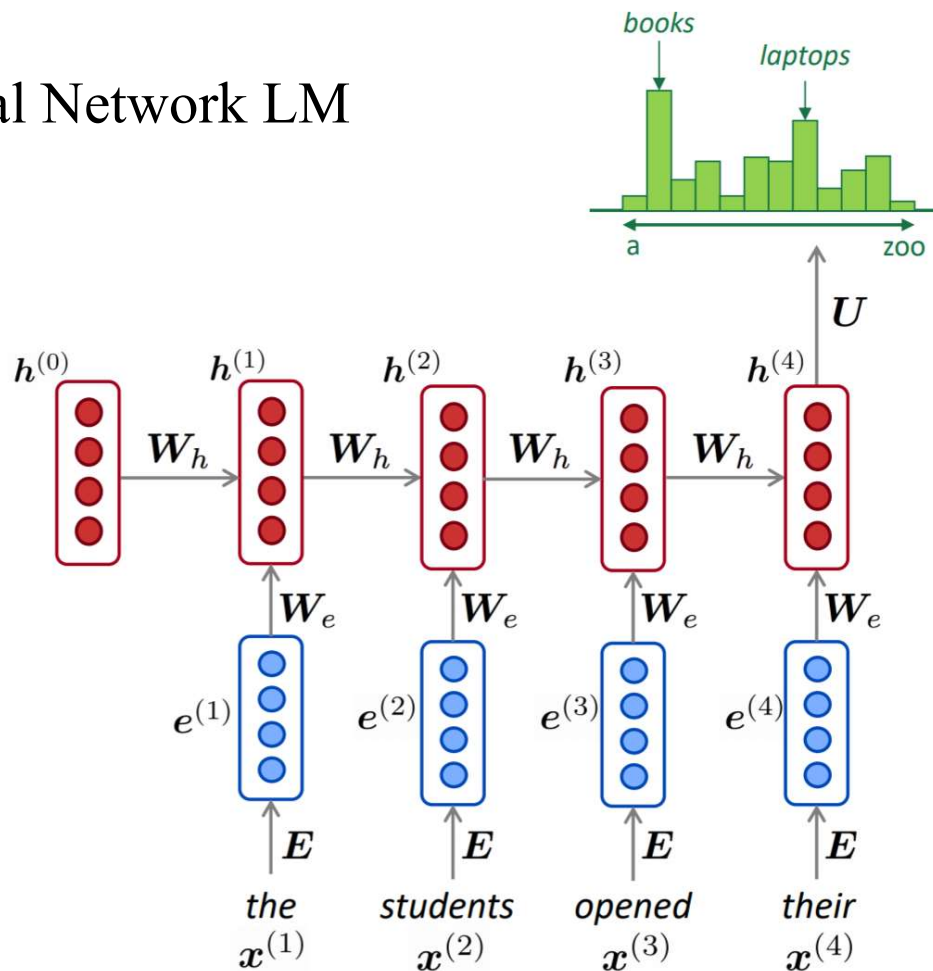


NNLM

- The word vector appears in two places
 - $C \in R^{D \times V}$
 - $U \in R^{h \times V}$, where h is the hidden size
- We can use not only left-context but also right-context
- Recurrent Neural Network LM

NNLM

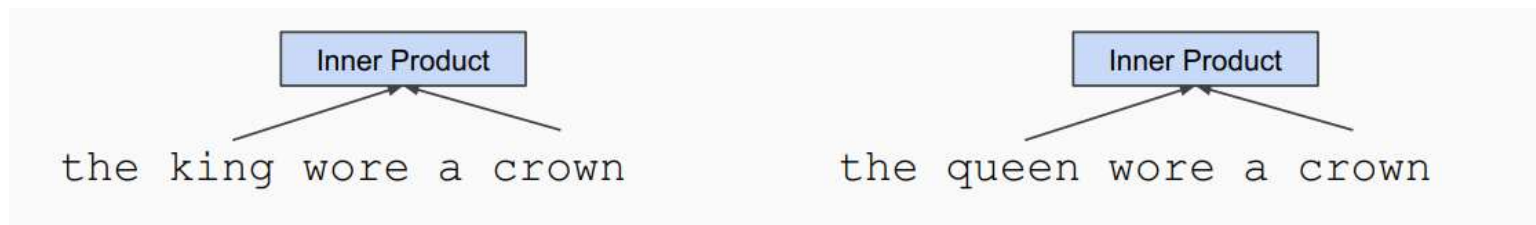
- Recurrent Neural Network LM





Word Embeddings

- Faster Version: C&W, word2vec, GloVe
 - pre-trained on text corpus from co-occurrence statistics

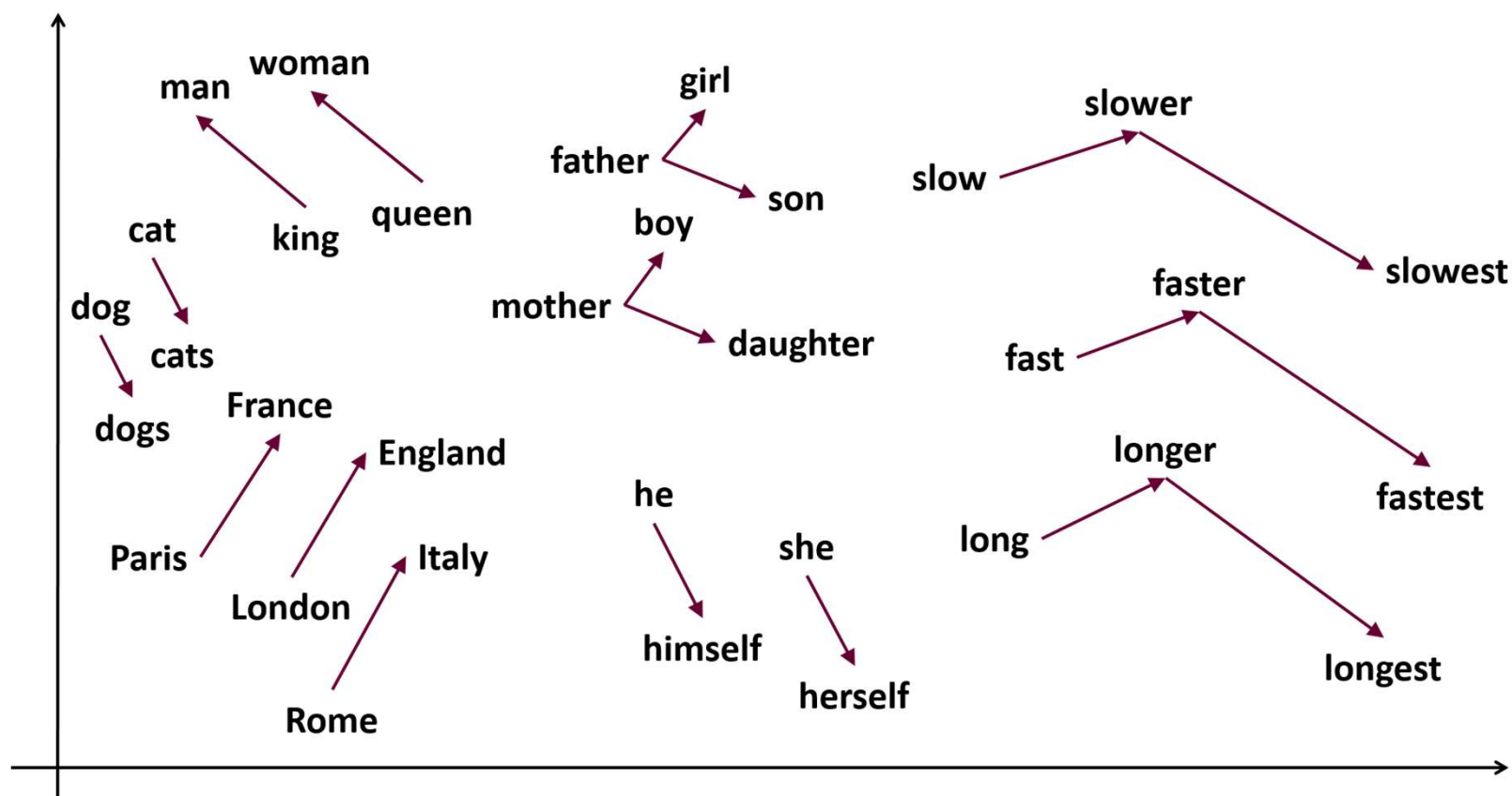


$$\text{King} - \text{Queen} = \text{Man} - \text{woman}$$



Word Embeddings

- Word2vec





Word Embeddings

- Word2vec
 - Low + Lower-high?

```
model.most_similar(positive=['low', 'lower'], negative=['high'], topn=10)
```

```
[('higher', 0.7889082431793213),  
 ('funnier', 0.6868950128555298),  
 ('greater', 0.6742820739746094),  
 ('More', 0.6719484329223633),  
 ('bigger', 0.665863573551178),  
 ('cheaper', 0.6584792137145996),  
 ('dumber', 0.6579981446266174),  
 ('quicker', 0.6447793245315552),  
 ('harder', 0.6422368288040161),  
 ('scarier', 0.6412309408187866)]
```



Word Embeddings

- Word2vec
 - Paris + France-Rome?

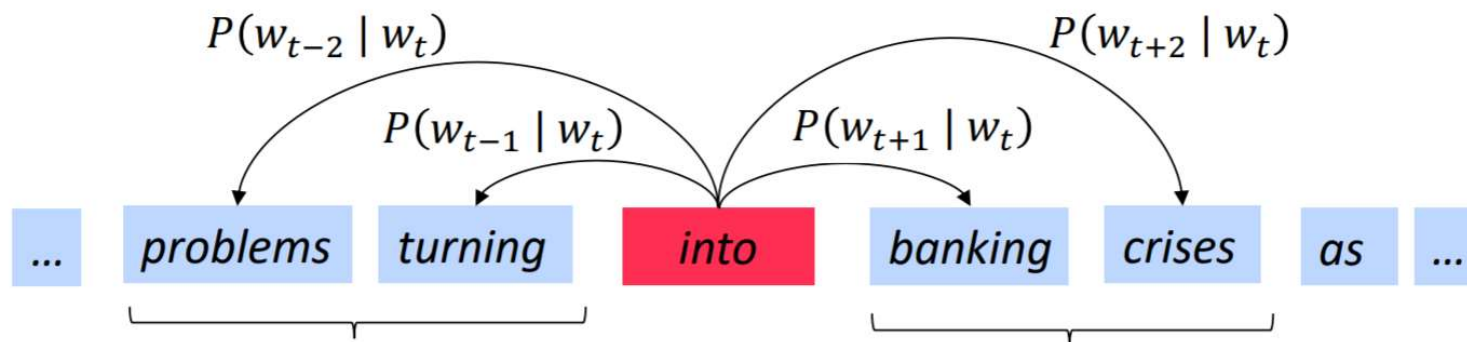
```
model.most_similar(positive=['Paris', 'France'], negative=['Rome'], topn=10)
```

```
[('Italy', 0.7843865752220154),  
 ('Germany', 0.7800211906433105),  
 ('England', 0.7736520171165466),  
 ('Japan', 0.7658747434616089),  
 ('America', 0.7626687288284302),  
 ('Africa', 0.7617301940917969),  
 ('Europe', 0.7588669061660767),  
 ('London', 0.7534858584403992),  
 ('India', 0.7437407970428467),  
 ('Mexico', 0.7425971031188965)]
```



Word Embeddings

- Word2vec: Skip-gram
 - Start with random word vectors
 - Iterate through each word in the whole corpus
 - Try to predict surrounding words using word vectors

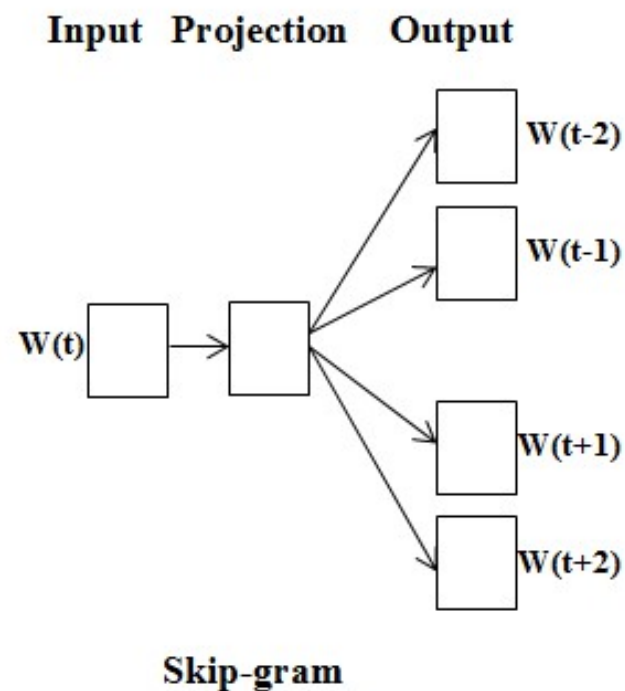
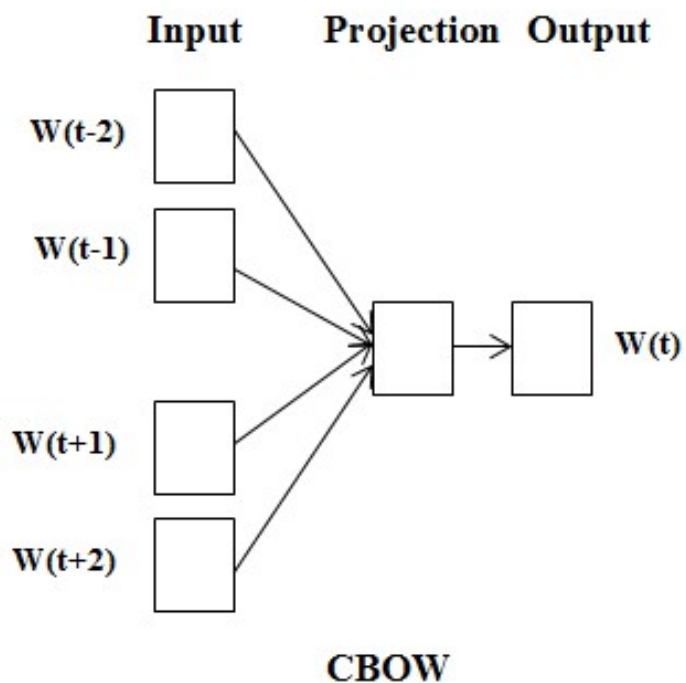


- $$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$
- Update vectors so you can predict better



Word Embeddings

- Word2vec: Skip-gram





Word2vec

- Optimization objective

- CBOW

$$P(w_t | \mathcal{C}_t) = \frac{\exp(\mathbf{v}_{\mathcal{C}_t} \cdot \mathbf{v}'_{w_t})}{\sum_{w' \in \mathbb{V}} \exp(\mathbf{v}_{\mathcal{C}_t} \cdot \mathbf{v}'_{w'})}$$

- Skip-Gram

$$P(c | w_t) = \frac{\exp(\mathbf{v}_{w_t} \cdot \mathbf{v}'_c)}{\sum_{w' \in \mathbb{V}} \exp(\mathbf{v}_{w_t} \cdot \mathbf{v}'_{w'})}$$

- Disadvantages:

- When the vocabulary is large and the computing resources are limited, the probability (normalization) computing efficiency is low.

Note: words and contexts use different vector matrices



Word2vec

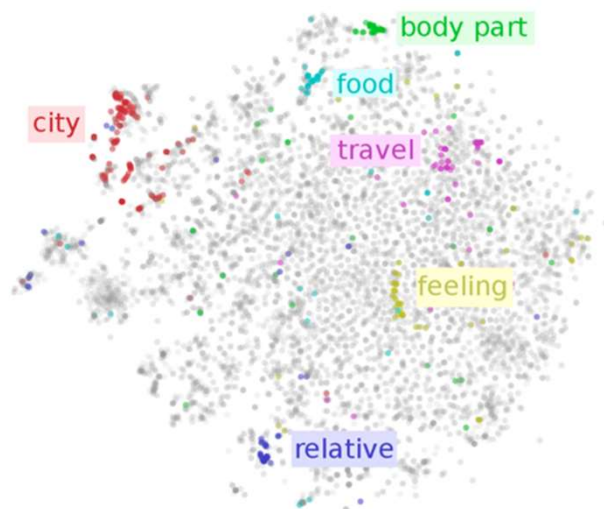
- Negative Sampling
 - Classify (words, contexts) in two ways: 1 means co occurrence within a given context, 0 means non co occurrence
 - Construct (word, context) negative examples by "changing words"

$$\log \sigma(\mathbf{v}_{w_t} \cdot \mathbf{v}'_{w_{t+j}}) + \sum_{i=1}^K \log \sigma(-\mathbf{v}_{w_t} \cdot \mathbf{v}'_{\tilde{w}_i})$$

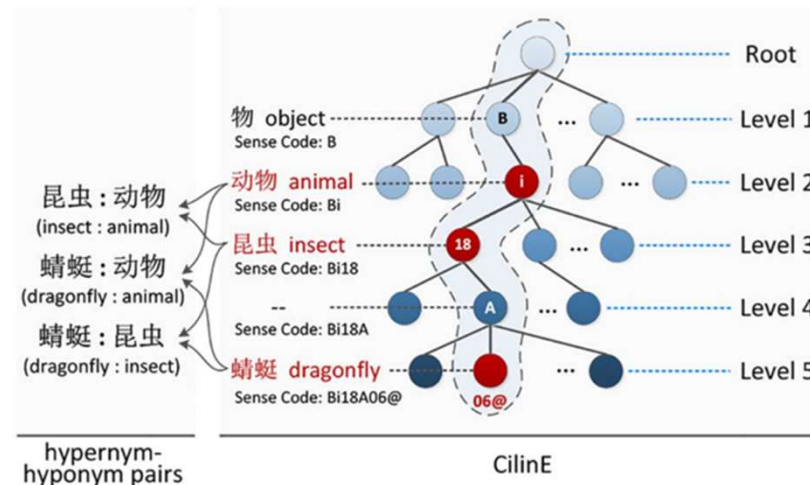
$\tilde{w}_i \sim P_n(w)$ (Negative sampling distribution)



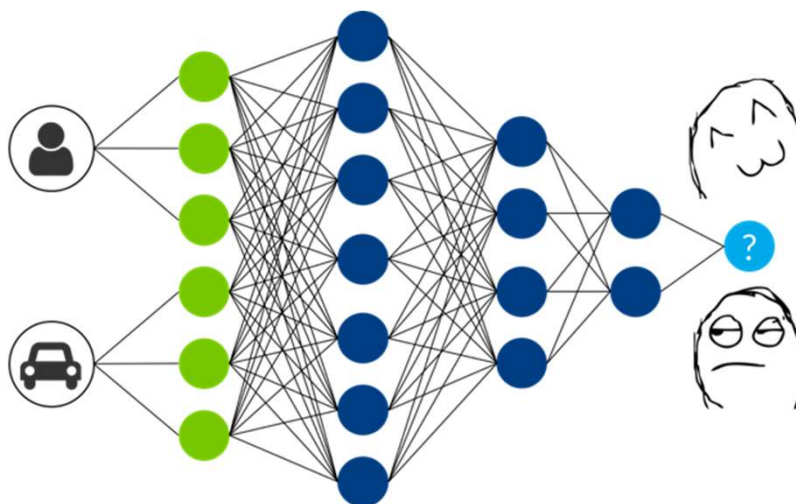
Application of Word2vec



Word Meaning Similarity Calculation



Supplement Knowledge Graph



Recommendation System



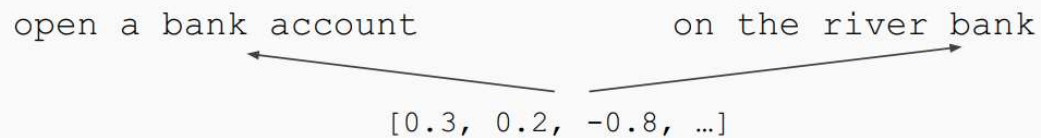
Word Embeddings

- Static Representation:
Word2vec, Glove, fastText
- These have two problems:
 - Always the same representation for a **word type** regardless of the context in which a **word token** occurs
 - We might want very fine-grained word sense disambiguation
 - We just have one representation for a word, but words have different **aspects**, including semantics, syntactic behavior, and register/connotations



Contextual Representations

- **Problem:** Word embeddings are applied in a context free manner



- **Solution:** Train *contextual* representations on text corpus

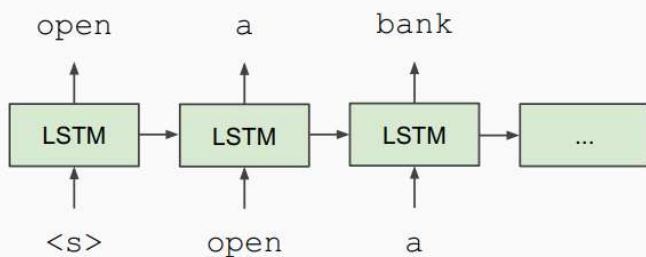




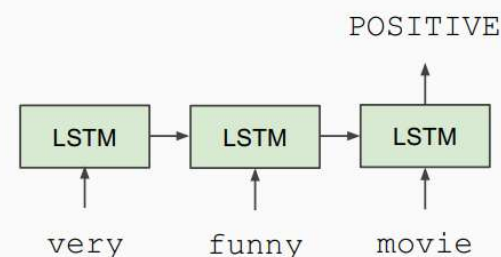
History of Contextual Representations

- *Semi-Supervised Sequence Learning, Google, 2015*

Train LSTM Language Model



Fine-tune on Classification Task

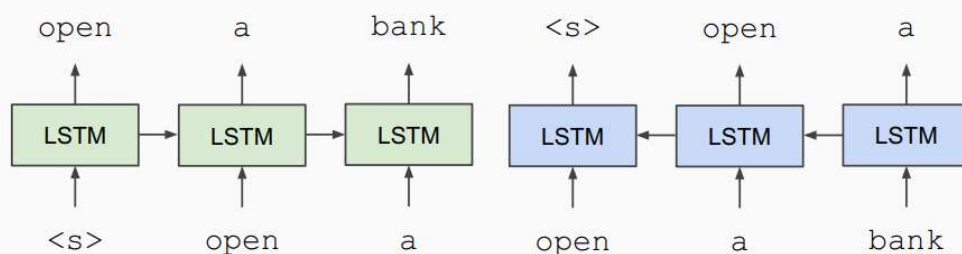




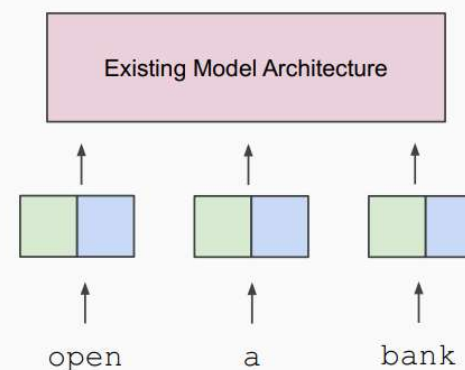
History of Contextual Representations

- *ELMo: Deep Contextual Word Embeddings*, AI2 & University of Washington, 2017

Train Separate Left-to-Right and Right-to-Left LMs



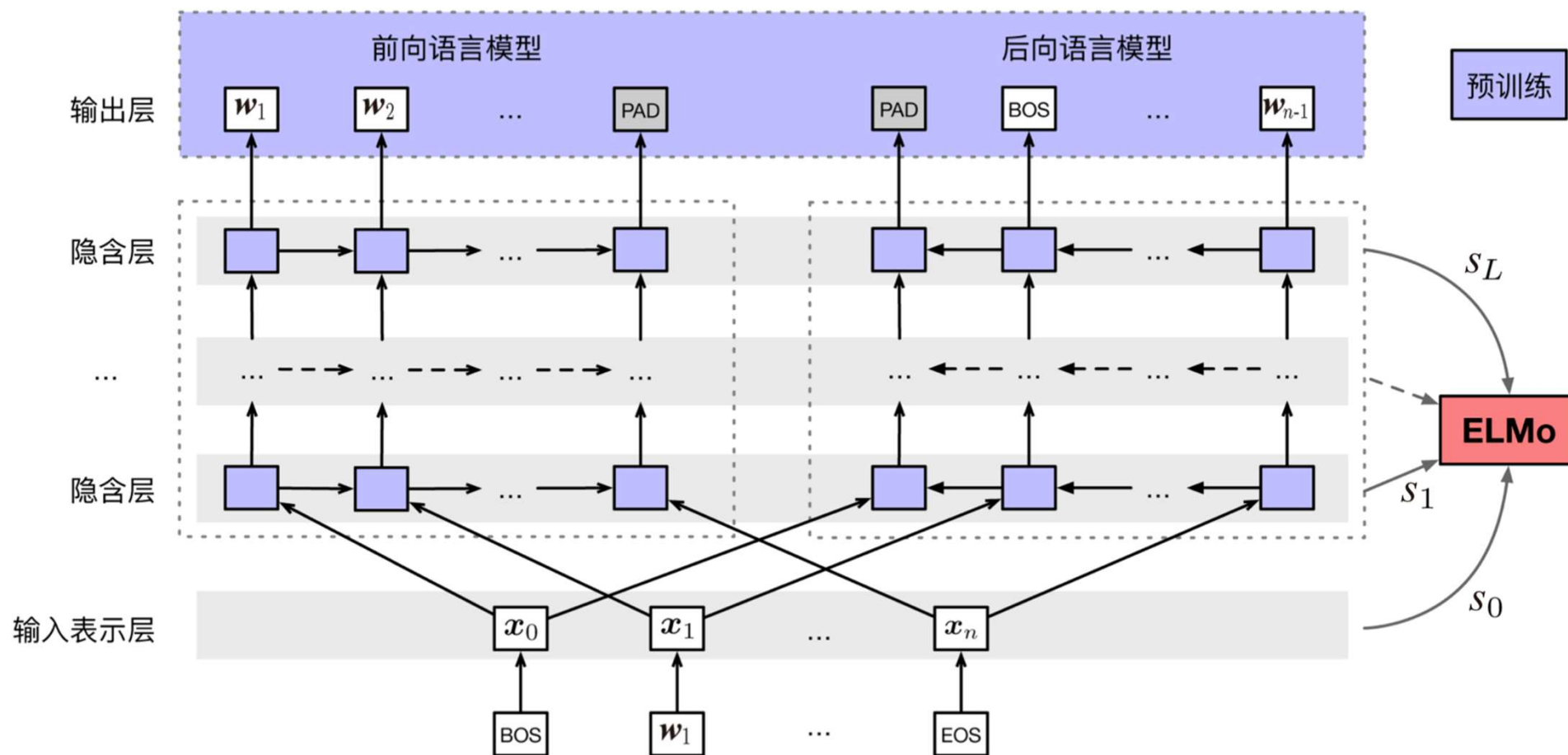
Apply as “Pre-trained Embeddings”



ELMo: Embeddings from Language Models



History of Contextual Representations





History of Contextual Representations

ELMo

- Train a bidirectional LM
- Aim at performant but not overly large LM:
 - Use 2 biLSTM layers
 - Use character CNN to build initial word representation (only)
 - 2048 char n-gram filters and 2 highway layers, 512 dim projection
 - Use 4096 dim hidden/cell LSTM states with 512 dim projections to next input
 - Use a residual connection
 - Tie parameters of token input and output (softmax) and tie these between forward and backward LMs



History of Contextual Representations

- ELMo learns task-specific combination of biLM representations
- This is an innovation that improves on just using top layer of LSTM stack

$$\begin{aligned} R_k &= \{\mathbf{x}_k^{LM}, \overrightarrow{\mathbf{h}}_{k,j}^{LM}, \overleftarrow{\mathbf{h}}_{k,j}^{LM} \mid j = 1, \dots, L\} \\ &= \{\mathbf{h}_{k,j}^{LM} \mid j = 0, \dots, L\}, \end{aligned}$$

$$\mathbf{ELMo}_k^{task} = E(R_k; \Theta^{task}) = \gamma^{task} \sum_{j=0}^L s_j^{task} \mathbf{h}_{k,j}^{LM}$$

- γ^{task} scales overall usefulness of ELMo to task;
- $\mathbf{s}^{task}$ are softmax-normalized mixture model weights



History of Contextual Representations

Apply ELMo

ELMo performs better in word sense disambiguation and nearest neighbor analysis than GloVe (static word vector)

模型	词	近邻
GloVe	play	playing, game, games, played, players, plays, player, Play, football, multiplayer
ELMo	Chico Ruiz made a spectacular <u>play</u> on Alusik's grounder . . .	Kieffer , the only junior in the group , was commended for his ability to hit in the clutch , as well as his all-round excellent <u>play</u>
	Olivia De Havilland signed to do a Broadway <u>play</u> for Garson . . .	. . . they were actors who had been handed fat roles in a successful <u>play</u> , and had talent enough to fill the roles competently , with nice understatement



History of Contextual Representations

Apply ELMo

- First run biLM to get representations for each word
- Then let (whatever) end-task model use them
 - Freeze weights of ELMo for purposes of supervised model
 - Concatenate ELMo weights into task-specific model
 - Details depend on task
 - Concatenating into intermediate layer as for TagLM is typical
 - Can provide ELMo representations again when producing outputs, as in a question answering system



History of Contextual Representations

Problems of Dynamic word vector

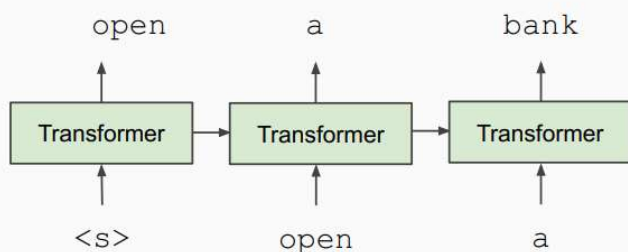
- Data
 - Training data is relatively limited
- Model
 - The number of parameters representing the model is relatively small (compared with the pre training language model), and the depth of the model is not enough
- Usage
 - When such a model is usually used, it means that the model itself does not participate in training (no weight update)
 - The representation model itself does not participate in training, which to some extent limits the generalization ability of the representation model on downstream tasks



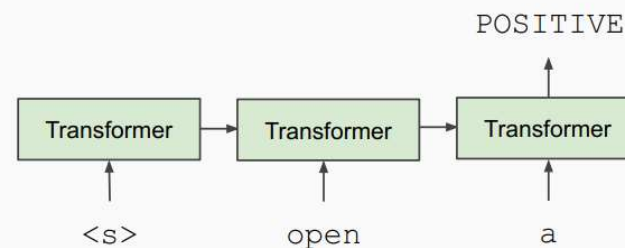
History of Contextual Representations

- *Improving Language Understanding by Generative Pre-Training*, OpenAI, 2018

**Train Deep (12-layer)
Transformer LM**

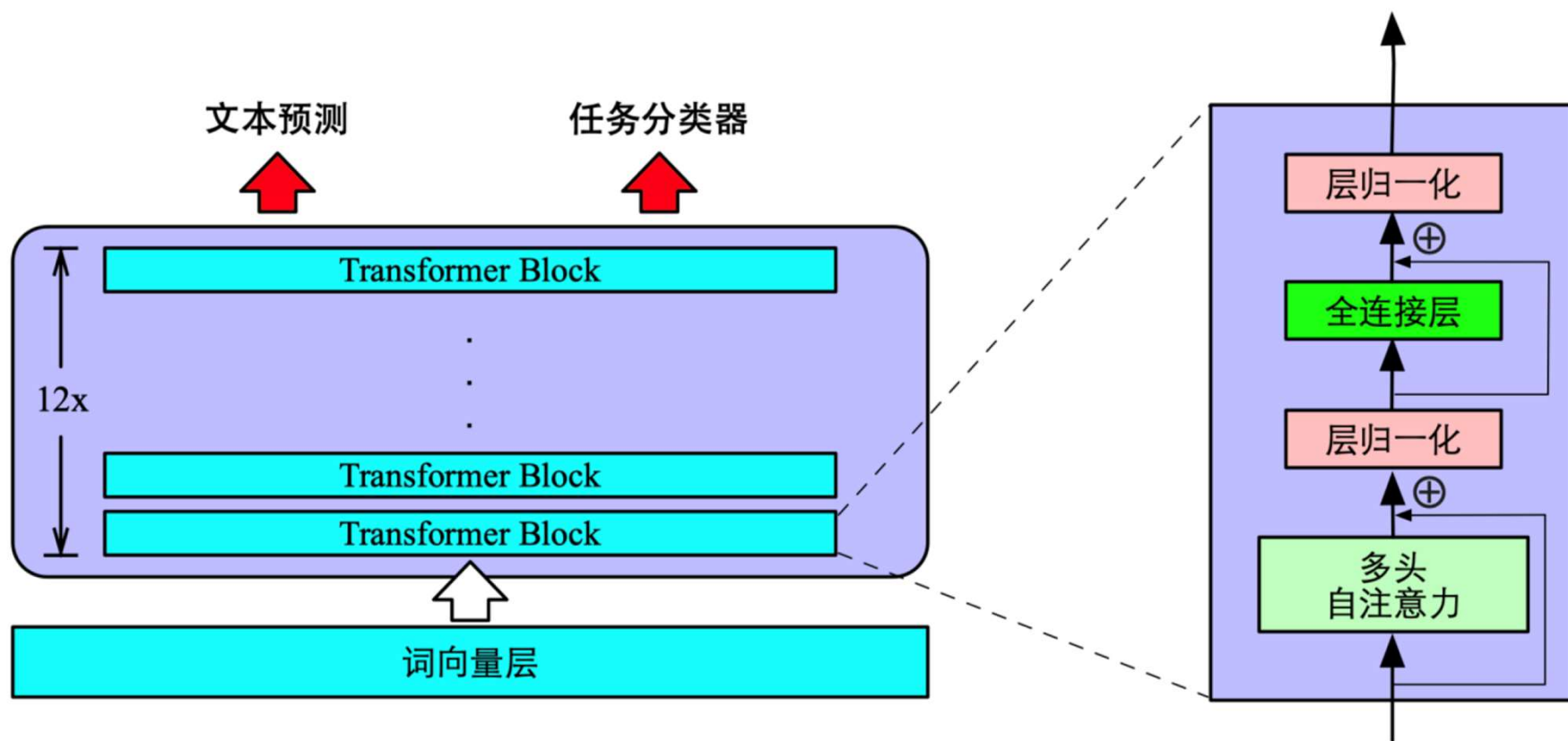


**Fine-tune on
Classification Task**





GPT





GPT

- Compute maximum likelihood estimation for a given text sequence (Unsupervised pre training)

$$\mathcal{L}^{\text{PT}}(x) = \sum_i \log P(x_i | x_{i-k} \cdots x_{i-1}; \boldsymbol{\theta})$$

- GPT uses multi-layer Transformer as the basic structure of the model

$$\mathbf{h}^{[0]} = \mathbf{e}_x \mathbf{W}^e + \mathbf{W}^p$$

$$\mathbf{h}^{[l]} = \text{Transformer-Block}(\mathbf{h}^{[l-1]}), \quad \forall l \in \{1, 2, \cdots, L\}$$

$$P(x) = \text{Softmax}(\mathbf{h}^{[L]} \mathbf{W}^e{}^\top)$$



Fine-tuning of GPT

- Use the representation of the last layer of GPT to complete relevant prediction tasks

$$P(y|x_1 \cdots x_n) = \text{Softmax}(\mathbf{h}^{[L]} \mathbf{W}^y)$$

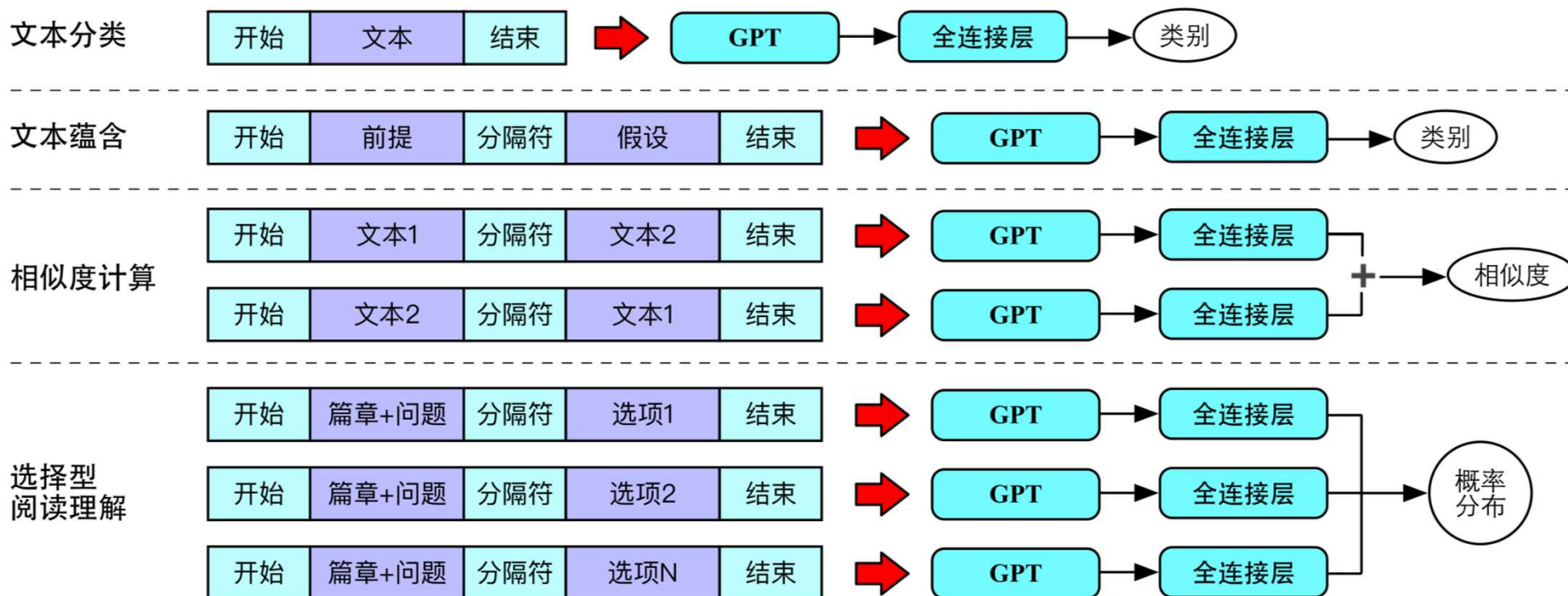
$$\mathcal{L}^{\text{FT}}(\mathcal{C}) = \sum_{(x,y)} \log P(y|x_1 \cdots x_n)$$

- In some cases, adding additional pre training losses can further improve performance

$$\mathcal{L}(\mathcal{C}) = \mathcal{L}^{\text{FT}}(\mathcal{C}) + \lambda \mathcal{L}^{\text{PT}}(\mathcal{C})$$



Fine-tuning of GPT

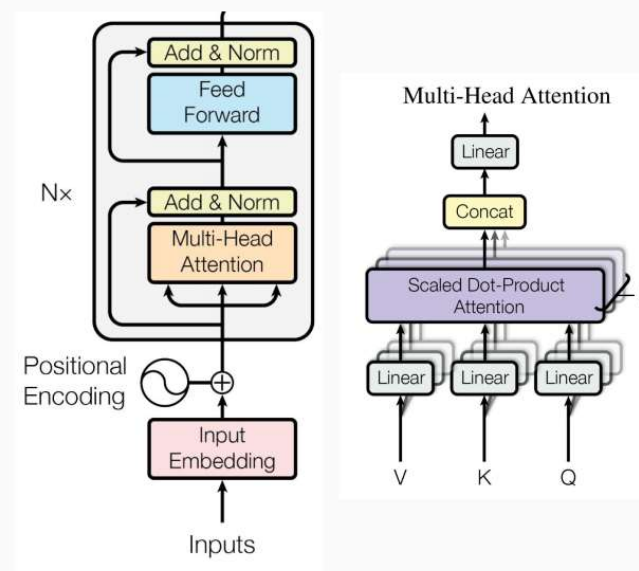




Transformer

Transformer encoder

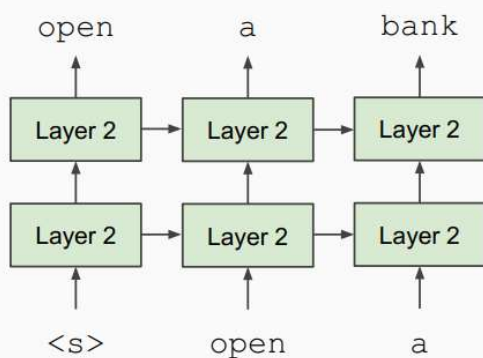
- Multi-headed self attention
 - Models context
- Feed-forward layers
 - Computes non-linear hierarchical features
- Layer norm and residuals
 - Makes training deep networks healthy
- Positional embeddings
 - Allows model to learn relative positioning



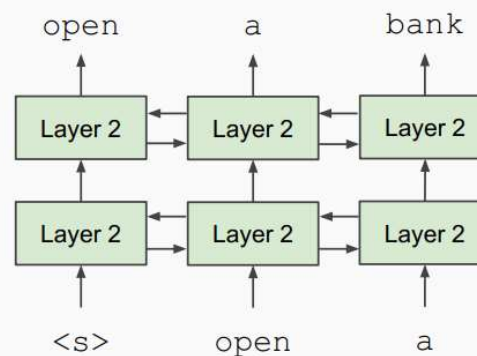


Bidirectional

Unidirectional context
Build representation incrementally



Bidirectional context
Words can "see themselves"





BERT: Masked LM

- **Solution:** Mask out $k\%$ of the input words, and then predict the masked words
 - We always use $k = 15\%$

the man went to the [MASK] to buy a [MASK] of milk

store gallon

↑ ↑

- Too little masking: Too expensive to train
- Too much masking: Not enough context



BERT: Masked LM

- Problem: Mask token never seen at fine-tuning
- Solution: 15% of the words to predict, but don't replace with [MASK] 100% of the time. Instead:
- 80% of the time, replace with [MASK]

went to the store → went to the [MASK]

- 10% of the time, replace random word

went to the store → went to the running

- 10% of the time, keep same

went to the store → went to the store

原文本	The man went to the store to buy some milk.
80% 概率替换为 [MASK]	The man went to the [MASK] to buy some milk.
10% 概率替换为随机词	The man went to the apple to buy some milk.
10% 概率保持原样	The man went to the store to buy some milk.



BERT: Masked LM

- Input layer

$$X = [\text{CLS}] x'_1 x'_2 \cdots x'_n [\text{SEP}]$$

$$\mathbf{v} = \text{InputRepresentation}(X)$$

- BERT coding layer

$$\mathbf{h} = \text{Transformer}(\mathbf{v})$$

- Output layer

$$P_i = \text{Softmax}(\mathbf{h}_i^{\text{m}} \mathbf{W}^{\text{t}\top} + \mathbf{b}^{\text{o}})$$



BERT: Next Sentence Prediction

- To learn *relationships* between sentences, predict whether Sentence B is actual sentence that proceeds Sentence A, or a random sentence

Sentence A = The man went to the store.
Sentence B = He bought a gallon of milk.
Label = IsNextSentence

Sentence A = The man went to the store.
Sentence B = Penguins are flightless.
Label = NotNextSentence



BERT: Next Sentence Prediction

- Input layer

$$X = [\text{CLS}] x_1^{(1)} x_2^{(1)} \cdots x_n^{(1)} [\text{SEP}] x_1^{(2)} x_2^{(2)} \cdots x_m^{(2)} [\text{SEP}]$$

$$\mathbf{v} = \text{InputRepresentation}(X)$$

- BERT coding layer

$$\mathbf{h} = \text{Transformer}(\mathbf{v})$$

- Output layer

$$P = \text{Softmax}(\mathbf{h}_0 \mathbf{W}^p + \mathbf{b}^o)$$



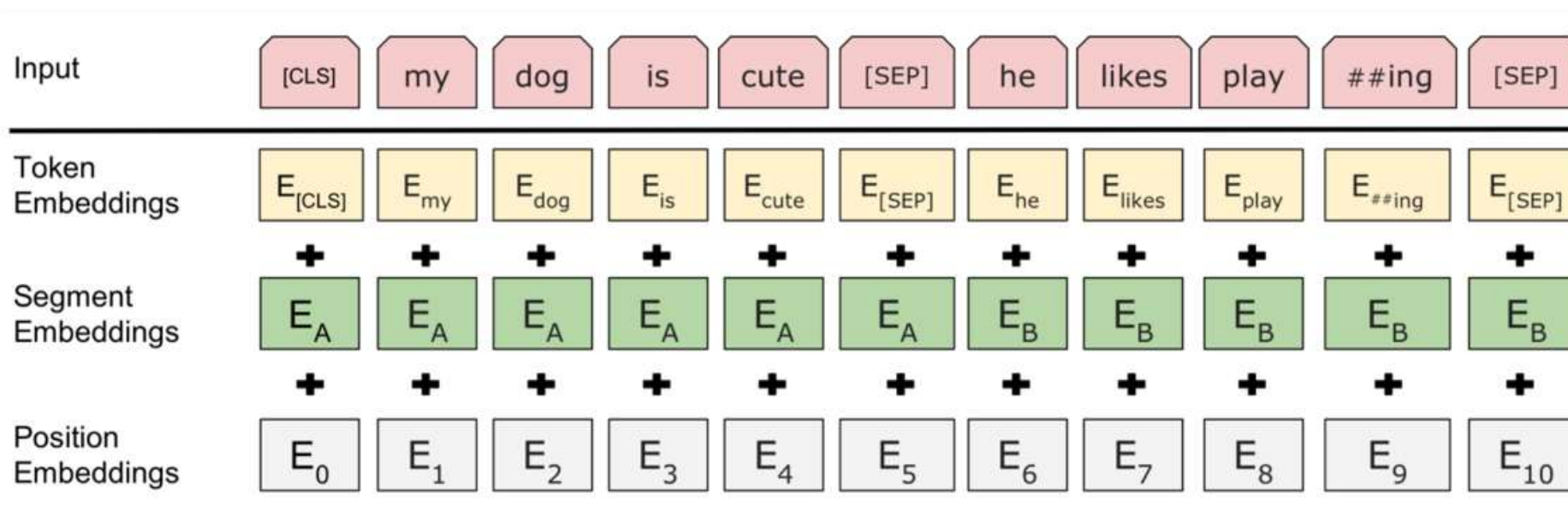
BERT: Model details

- Data: Wikipedia (2.5B words) + BookCorpus (800M words)
- Batch Size: 131,072 words (1024 sequences * 128 length or 256 sequences * 512 length)
- Training Time: 1M steps (~40 epochs)
- Optimizer: AdamW, 1e-4 learning rate, linear decay
- BERT-Base: 12-layer, 768-hidden, 12-head
- BERT-Large: 24-layer, 1024-hidden, 16-head
- Trained on 4x4 or 8x8 TPU slice for 4 days



BERT: Model details

Bert Sentence Pair Encoding



Token embeddings are word pieces

Learned segmented embedding represents each sentence

Positional embedding is as for other Transformer architectures

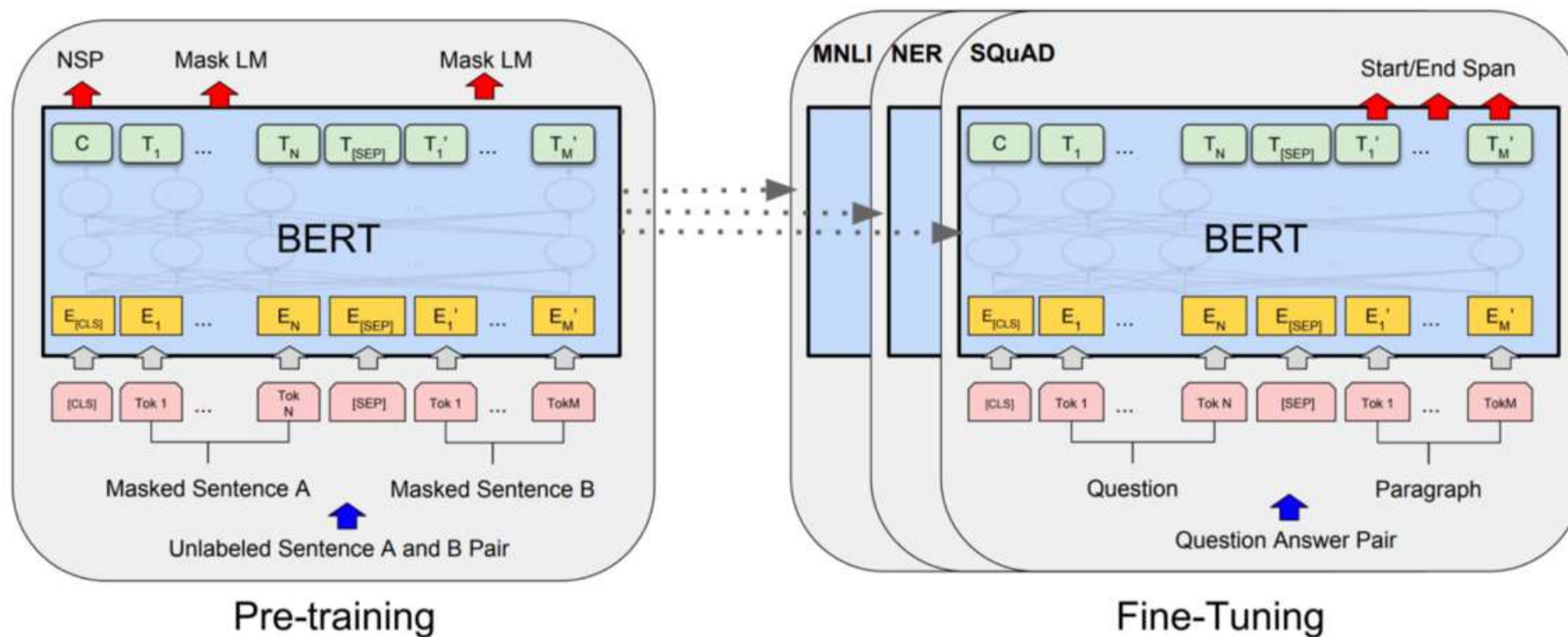


BERT: Model details

- Transformer encoder (as before)
- Self-attention $\Rightarrow$ no locality bias
 - Long-distance context has “equal opportunity”
- Single multiplication per layer $\Rightarrow$ efficiency on GPU/TPU
- Train on Wikipedia + BookCorpus
- Train 2 model sizes:
 - BERT-Base: 12-layer, 768-hidden, 12-head
 - BERT-Large: 24-layer, 1024-hidden, 16-head
- Trained on 4x4 or 8x8 TPU slice for 4 days

BERT: Fine Tuning

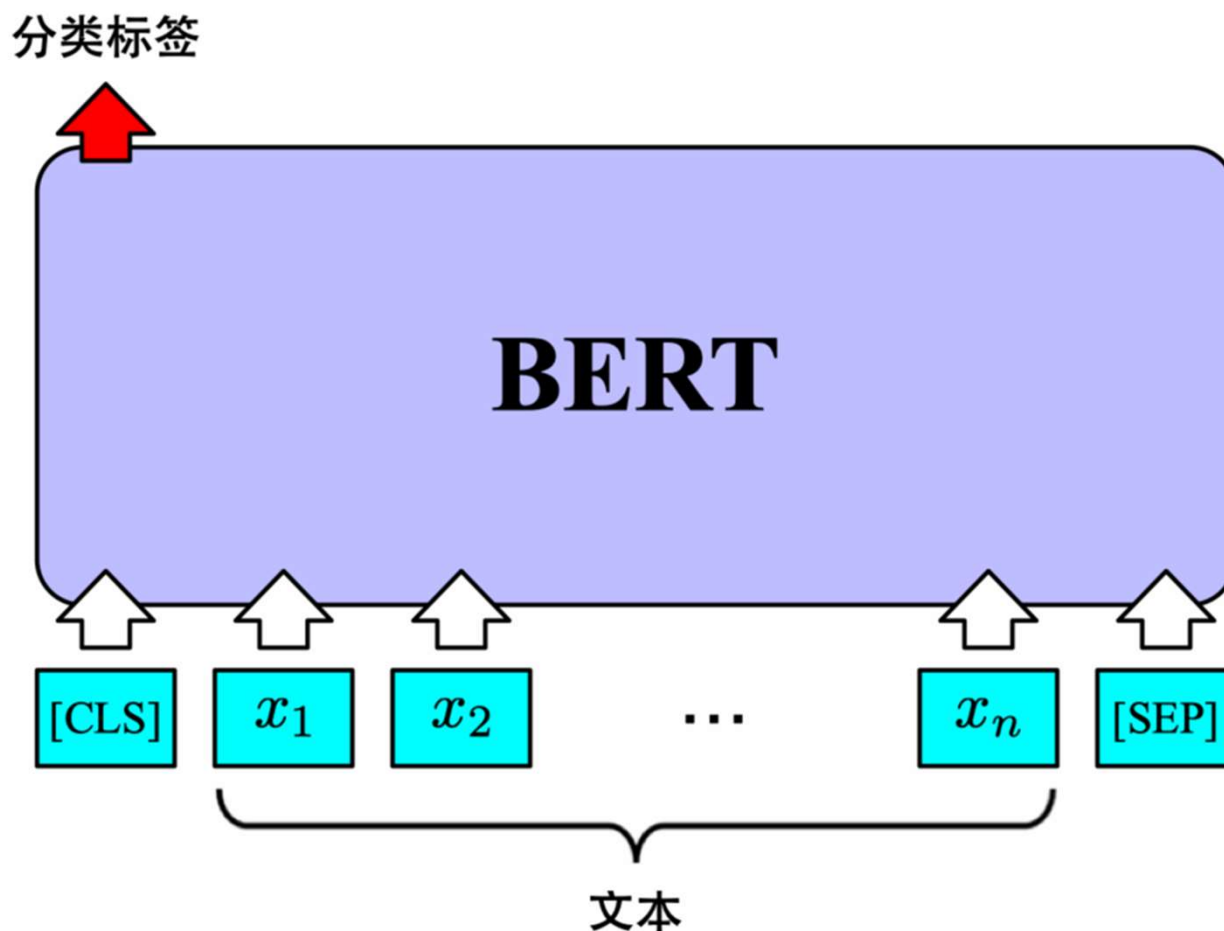
- Simply learn a classifier built on the top layer for each task that you fine tune for





BERT for Text Classification

- Utilizing BERT for Single Sentence Classification





BERT for Text Classification

- Input layer

$$X = [\text{CLS}] x_1 x_2 \cdots x_n [\text{SEP}]$$

$$\mathbf{v} = \text{InputRepresentation}(X)$$

- BERT coding layer

$$\mathbf{h} = \text{BERT}(\mathbf{v})$$

- Output layer

$$P = \text{Softmax}(\mathbf{h}_0 \mathbf{W}^o + \mathbf{b}^o)$$



BERT for Reading Comprehension

- Utilizing BERT for Reading Comprehension

【篇章】

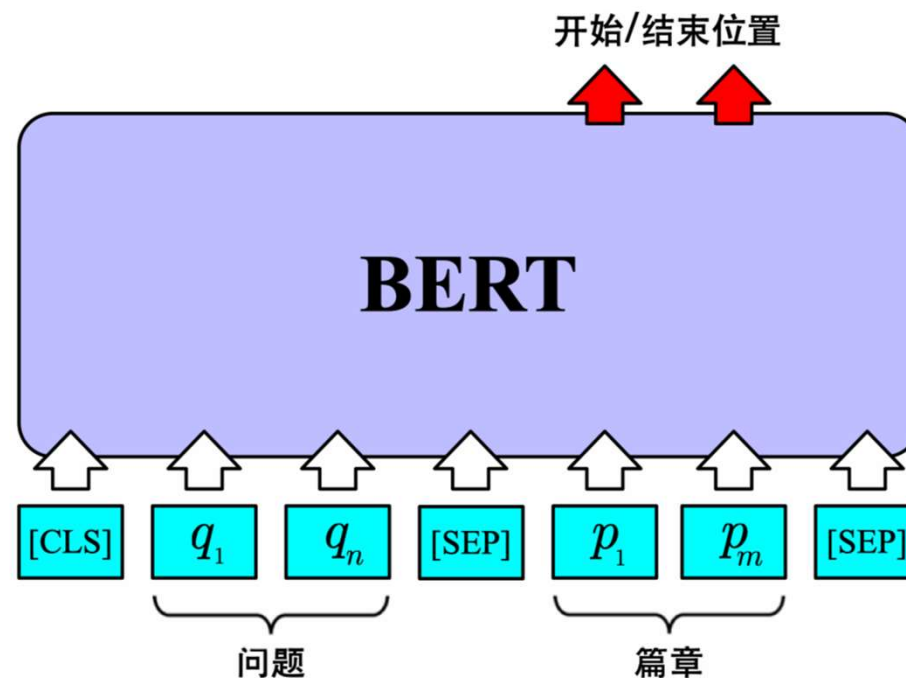
哈尔滨工业大学（简称哈工大）隶属于工业和信息化部，以理工为主，理工管文经法艺等多学科协调发展，拥有哈尔滨、威海、深圳三个校区。学校始建于1920年，1951年被确定为全国学习国外高等教育办学模式的两所样板大学之一，1954年进入国家首批重点建设的6所高校行列，曾被誉为工程师的摇篮。学校于1996年进入国家“211工程”首批重点建设高校，**1999年**被确定为国家首批“985工程”重点建设的9所大学之一，2000年与同根同源的哈尔滨建筑大学合并组建新的哈工大，2017年入选“双一流”建设A类高校名单。

【问题】

哈尔滨工业大学哪一年入选国家首批“985工程”？

【答案】

1999年





BERT for Reading Comprehension

- Input layer

$$X = [\text{CLS}] q_1 q_2 \cdots q_n [\text{SEP}] p_1 p_2 \cdots p_m [\text{SEP}]$$

$$\mathbf{v} = \text{InputRepresentation}(X)$$

- BERT coding layer

$$\mathbf{h} = \text{BERT}(\mathbf{v})$$

- Output layer

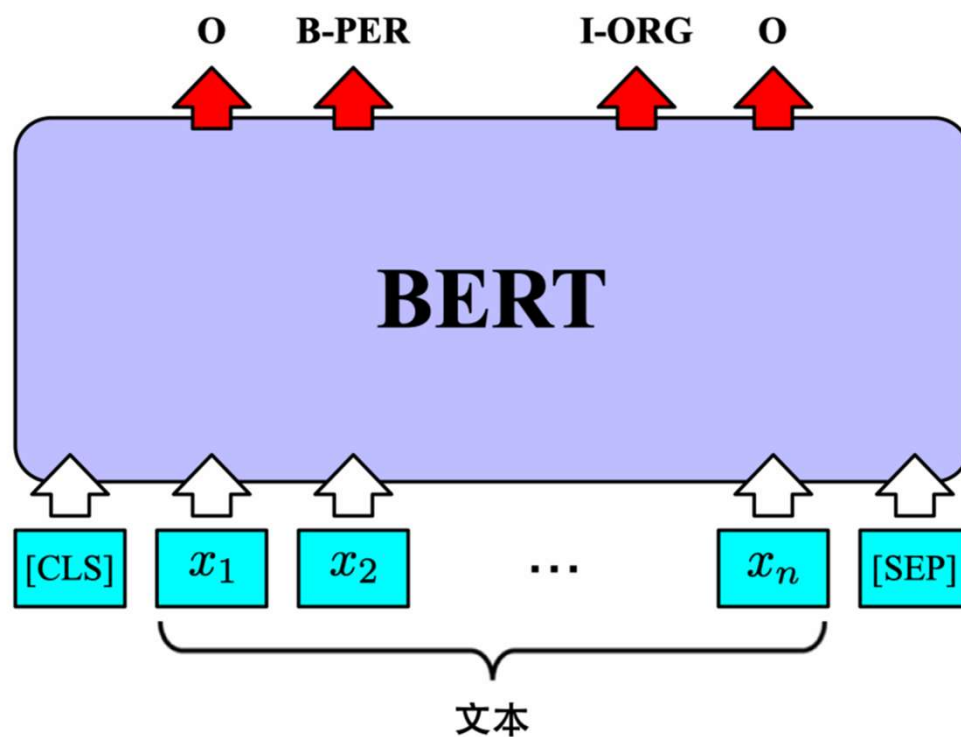
$$P^e = \text{Softmax}(\mathbf{h}\mathbf{W}^e + b^e)$$



BERT for Sequence Tagging

- Utilizing BERT for Sequence Tagging

原始标签	B-PER	I-PER	O	O	O	O	B-LOC	
原始输入	John	Smith	has	never	been	to	Harbin	
处理后的标签	B-PER	I-PER	O	O	O	O	B-LOC	B-LOC
处理后的输入	John	Smith	has	never	been	to	Ha	##rbin





BERT for Text Classification

- Input layer

$$X = [\text{CLS}] x_1 x_2 \cdots x_n [\text{SEP}]$$

$$\mathbf{v} = \text{InputRepresentation}(X)$$

- BERT coding layer

$$\mathbf{h} = \text{BERT}(\mathbf{v})$$

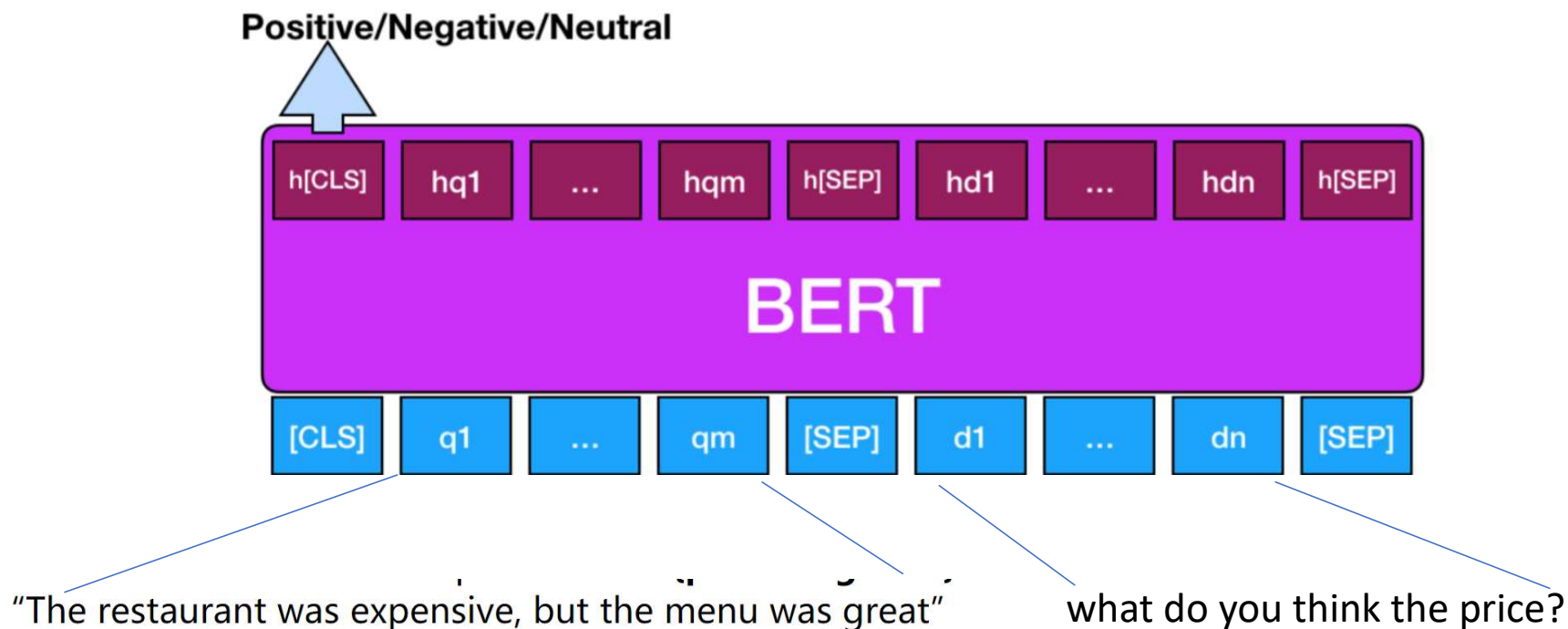
- Output layer

$$P_t = \text{Softmax}(\mathbf{h}_t \mathbf{W}^o + \mathbf{b}^o), \quad \forall t \in \{1, \cdots, N\}$$



BERT for ABSA

- Utilizing BERT for Aspect-Based Sentiment Analysis



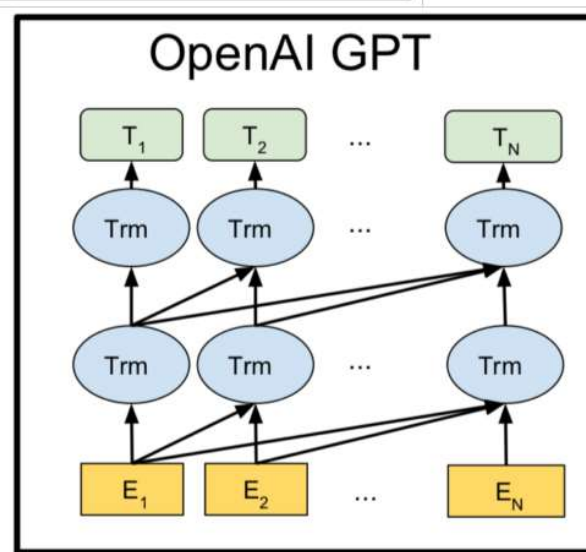
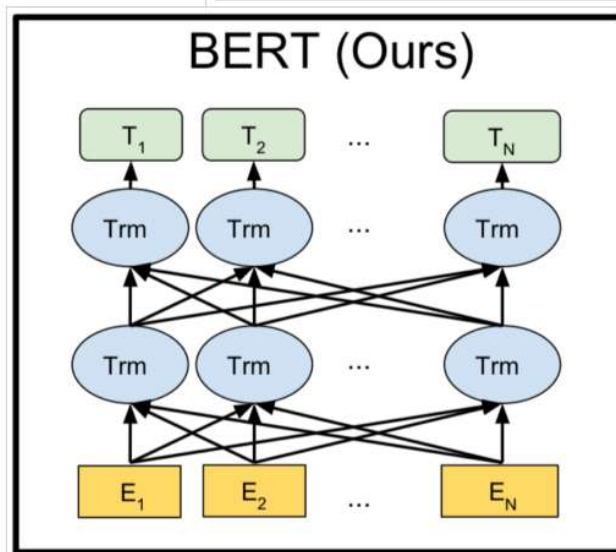
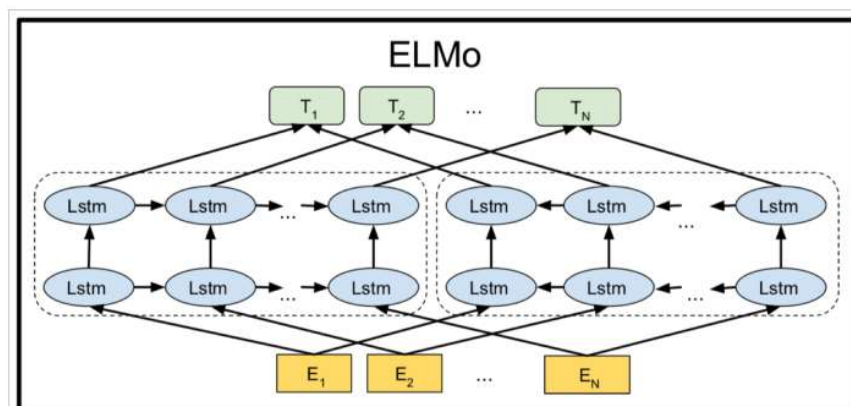


BERT for ABSA

- Utilizing BERT for Aspect-Based Sentiment Analysis

Models	4-way	3-way	Binary
XRCE	78.1	-	-
NRC-Canada	82.9	-	-
LSTM	-	82.0	88.3
ATAE-LSTM	-	84.0	89.9
BERT-single	83.7	86.9	93.3
BERT-pair-QA-M	85.2	89.3	95.4
BERT-pair-NLI-M	85.1	88.7	94.4
BERT-pair-QA-B	85.9	89.9	95.6
BERT-pair-NLI-B	84.6	88.7	95.1

ELMo, BERT, GPT





GLUE Results

- Remarkable!

GLUE Results

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.9	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	88.1	91.3	45.4	80.0	82.3	56.0	75.2
BERT _{BASE}	84.6/83.4	71.2	90.1	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	91.1	94.9	60.5	86.5	89.3	70.1	81.9



Post-BERT Pre-training

- RoBERTa: train BERT for more epochs and on more data
- XLNet: Permutation Language Modeling instead of MLM
- ALBERT: Cross-layer parameter sharing
- T5
 - bigger model size: 11B parameters (BERT-Large is 330M)
 - more data: 120B words
- ELECTRA: Train model to discriminate locally plausible text from real text



Baidu ERNIE 1.0

- Enhanced Representation from kNowledge IntEgration
 - knowledge masking strategies

Learned by BERT



Learned by ERNIE



哈尔滨是黑龙江的省会, 国际冰雪文化名城



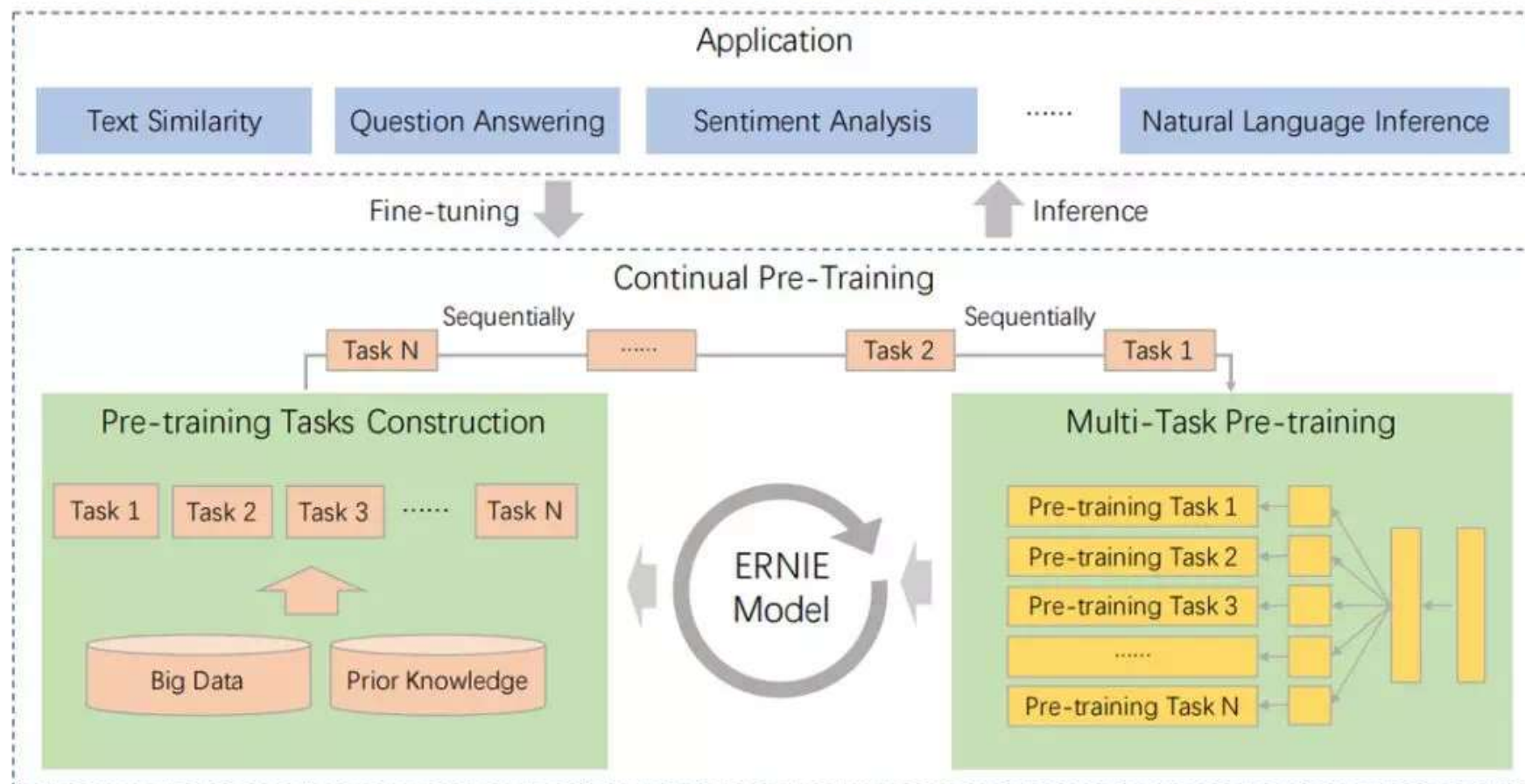
Baidu ERNIE 2.0

- Enhanced Representation from kNowledge IntEgration 2.0
 - word-aware pre-training tasks
 - structure-aware pre-training tasks
 - e.g. sentence reordering task
 - semantic-aware pre-training tasks
 - e.g. discourse relation task



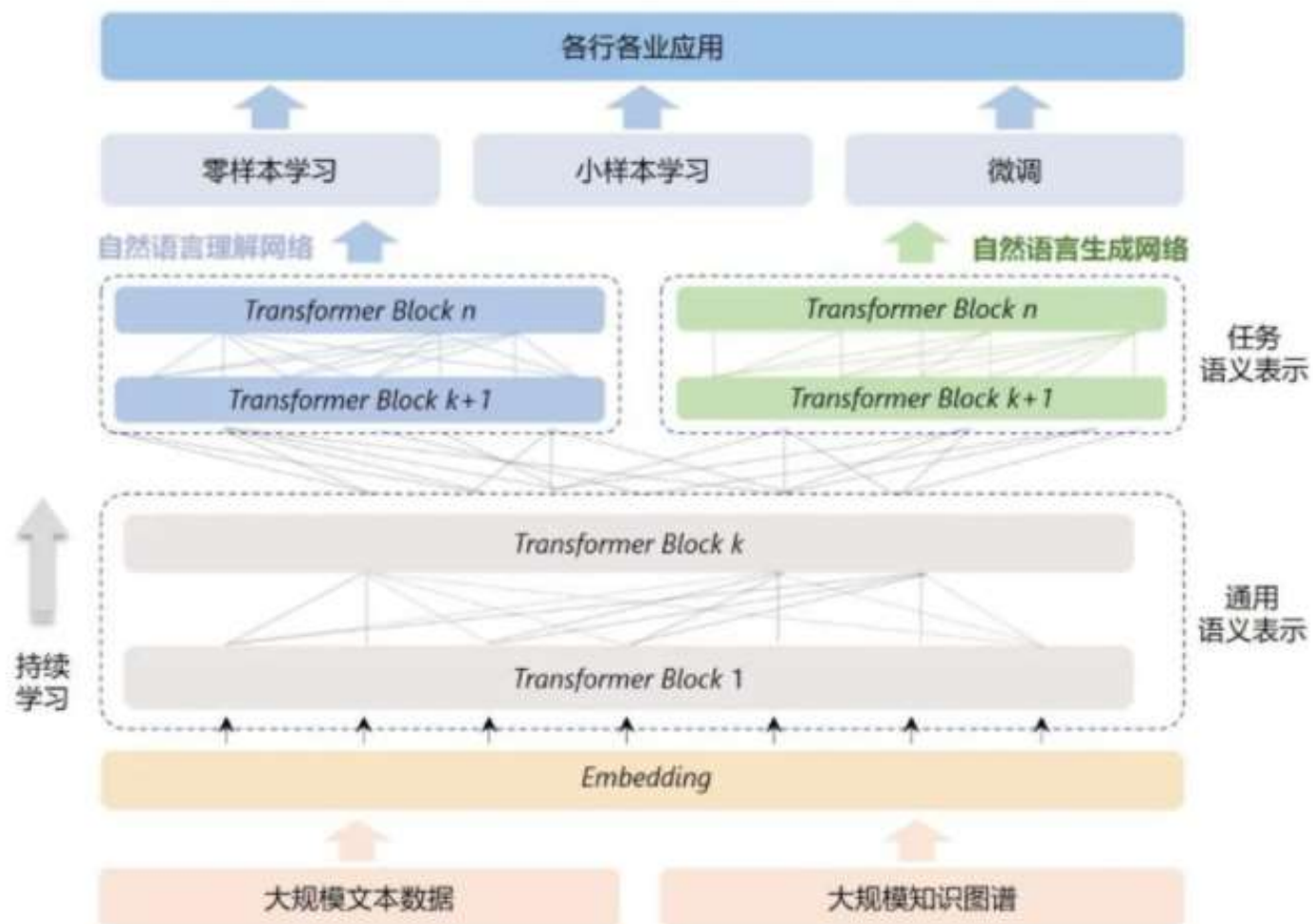
Baidu ERNIE 2.0

ERNIE 2.0 : A Continual Pre-training framework for Language Understanding





Baidu ERNIE 3.0





Baidu ERNIE 3.0

古文创作

输入古文:

江西孟龙潭与朱孝廉客都中,偶涉一兰若,殿宇禅舍,俱不甚弘敞,惟一老僧挂搭其中。见客入,肃衣出迓,导与随喜。殿中塑志公像,两壁画绘精妙,人物如生。东壁画散花天女,内一垂髻者,拈花微笑,樱唇欲动,眼波将流。

ERNIE 3.0:

西壁画一小沙弥,面目奇古,双手合十,似在祈祷。客大骇,请问之。僧曰:“此乃佛经也。”客曰:“此何所谓?”僧曰:“此有佛性,无所不有。”客曰:“何以见之?”僧曰:“佛性即是佛性。佛性无所不有,所以不能言说。”客大悟,遂去。

歌词创作

输入歌词:

早知惊鸿一场,何必情深一往,昨日人去楼空泪微凉,道不尽缘本无常,

ERNIE 3.0:

情难断,心难了,爱难忘,情难求,人难舍。

对对联

输入上联:

口十心思,思乡,思友,思父母。

ERNIE 3.0:

眼四面观,观景,观人,观自己。



GPT3

- GPT3
 - model size: 175B
 - training data: 45TB
 - train time: 3640pfs-day
 - train cost: \$12 million





GPT3

- Write novel/article
- Generate jokes

The importance of being on twitter

by Jerome K. Jerome
London, Summer 1897

It is a curious fact that the last remaining form of social life in which the people of London are still interested is Twitter. I was struck with this curious fact when I went on one of my periodical holidays to the sea-side, and found the whole place twittering like a starling-cage. I called it an anomaly, and it is.

I spoke to the sexton, whose cottage, like all sexton's cottages, is full of antiquities and interesting relics of former centuries. I said to him, "My dear sexton, what does all this twittering mean?" And he replied, "Why, sir, of course it means Twitter." "Ah!" I said, "I know about that. But what is Twitter?"

"It is a system of short and pithy sentences strung together in groups, for the purpose of conveying useful information to the initiated, and entertainment and the exercise of wits to the initiated, and entertainment and the exercise of wits to the rest of us."

"Very interesting," I said. "Has it a name?"

"It has," he said; "it is called Twitter."

"Yes," I said, "I know that, but what is it?"

"It is a system of information," he said.

"Oh, yes," I replied; "but what is it?"

"Why, sir," he said, "you can go up to any of the gentlemen you see twittering in the street, and say to him, 'You are a fool,' or 'Your wife is an adulteress,' or 'You have stolen that hat,' and if he is a member of the initiated he will answer you in the same form and tell you that you are a liar, or that your eyes resemble the eyes of a duck, or that you have stepped out of your part in the last charade you acted in, or that you were for a short time a statistician in a Government Office, and he will go on to tell you the whole story of your life, in language so exceedingly small and pointed that even you will be glad you can't understand it."



GPT3

- Generate webpage code

Describe a layout.

Just describe any layout you want, and it'll try to render below!

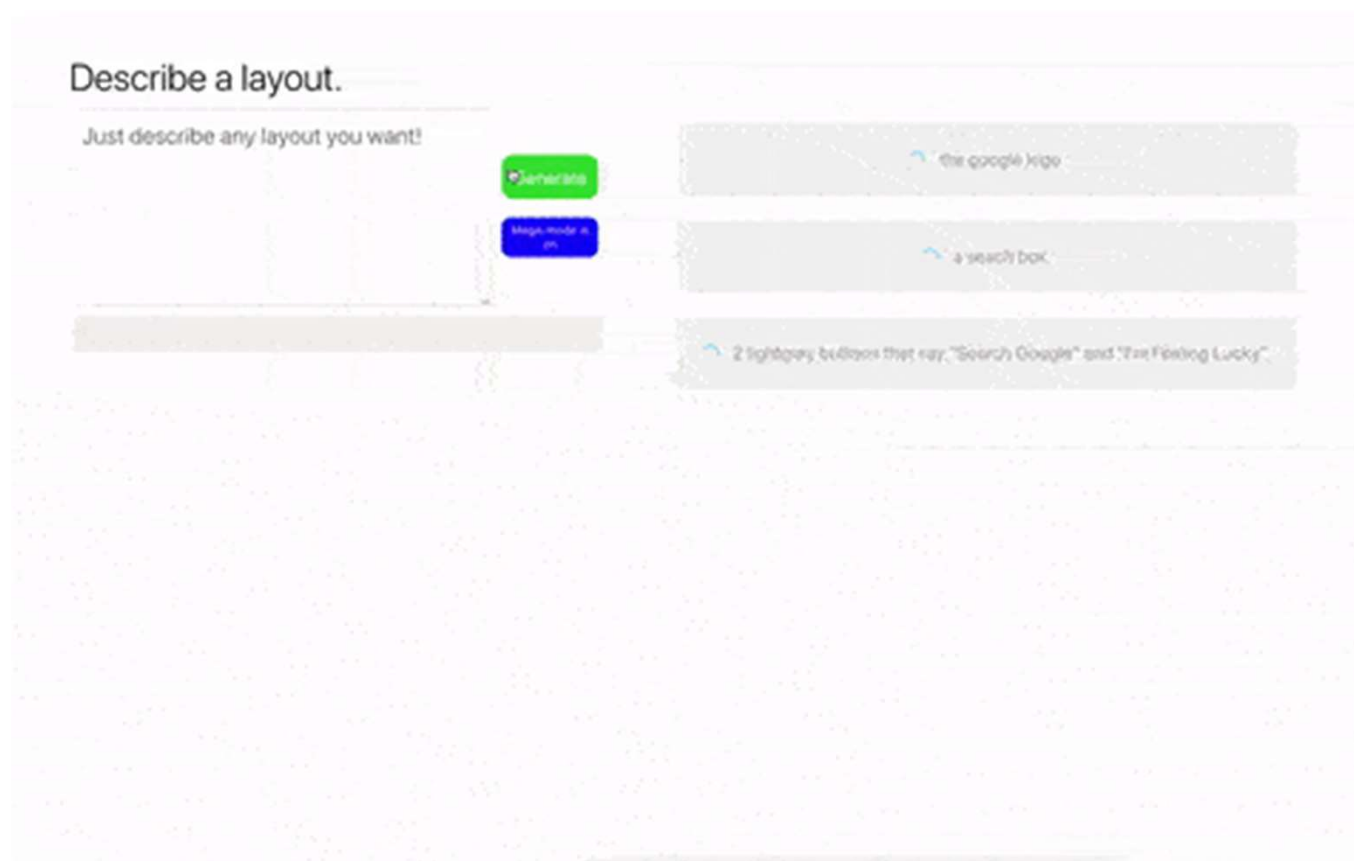
A div that contains 3 buttons each with a random color.

Generate



GPT3

- Generate webpage code





GPT3

- English to shell

 [HOME](#) [DOCUMENTATION](#) [PLAYGROUND](#) davinci ▾ [Logout](#)

Playground ⓘ ⚙ Text to command ✕ ▾

Q: List files in this directory, including hidden files
A: `ls -a`

Q: Go to the home directory
A: `cd ~`

Q: Move cat.jpg to my Desktop
A: `mv cat.jpg ~/Desktop`

Q: List running processes
A: `ps aux`

Q: find occurrences of the string "pepsi" in every file in the current directory recursively
A: `grep -r "pepsi" *`

Q: copy the file testing.txt onto a remote server at remoteserver.com
A: `scp testing.txt remoteserver.com:~/`

Q: start the node program at index.js
A: `node index.js`

Q: start a simple web server with python
A: `python -m SimpleHTTPServer`

Q: install the npm package eslint as a dev dependency
A: `npm install eslint --save-dev`

Q: display the file testing.txt in the console
A: `cat testing.txt`

SUBMIT ➡ ☒ Inject structural text (start sequence, reset sequence) 



GPT3

- poor common sense understanding

Q: Which is heavier, a toaster or a pencil?

A: A pencil is heavier than a toaster.

Q: How many eyes does a giraffe have?

A: A giraffe has two eyes.

Q: How many eyes does my foot have?

A: Your foot has two eyes.

Q: How many eyes does a spider have?

A: A spider has eight eyes.

Q: How many eyes does the sun have?

A: The sun has one eye.

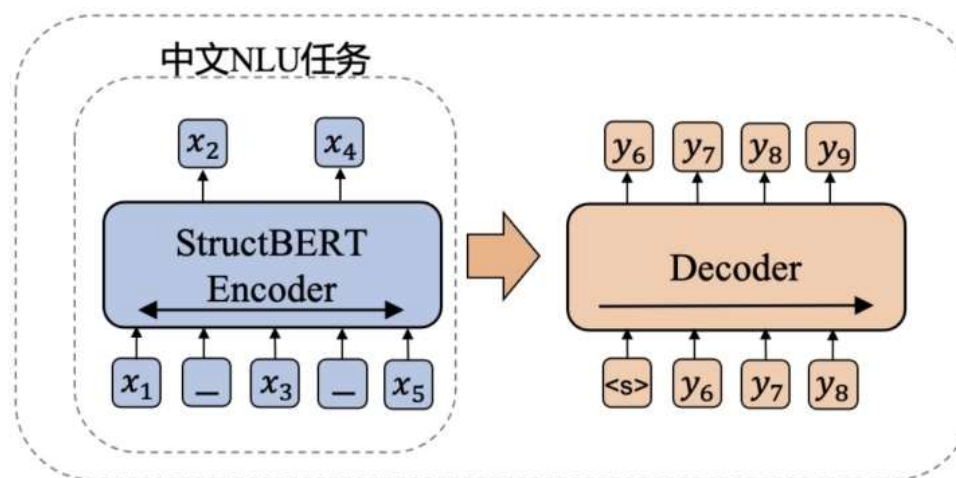
Q: How many eyes does a blade of grass have?

A: A blade of grass has one eye.



PLUG: Pre-training for Language Understanding and Generation

- Bidirectional modeling method of encoder decoder
- Firstly, a standard structBERT model with 24 layers / 8192 hidden size is trained as the encoder
- Then, the encoder is used to generate the initialization of the model, and a 6 layers / 8192 hidden size decode is plug-in





PLUG: Pre-training for Language Understanding and Generation

::: 小说仿写 :::

前文：黛玉只就宝玉手中看了一看，便问道：“还是单送我一人的，还是别的姑娘们都有呢？”周瑞家的道：“各位都有了，这两支是姑娘的了。”

AI续写：黛玉却想道：“我如今是个孤女，他们送的那些东西，还有什么可看的。”因此自己心下不乐，只是独自一个人在那里摆弄那些物事。

::: 诗歌生成 :::

前文：人闲桂花落，夜静春山空。月出惊山鸟，

AI续写：林深闻杜鹃。谁家竹径深，何处禅房幽。山光映疏影，水色接平波。千载空悠哉，万载自逍遥。松间风入耳，泉眼月浮光。

::: 自由创作 :::

前文：30年后，人类走出了太阳系，发现了一个被冰川覆盖的星球，

AI续写：这里居住着一个神秘而强大的部族。他们在这里建立了一个叫做冰川神殿的国度。30年后，一个叫做冰川神王的部族在一次探险中发现了这个星球上唯一的一块大陆冰川谷。于是，他们派出了十几名精英去寻找冰川谷。经过艰苦的探索，他们终于在冰川谷的冰川深处发现了一个巨大的冰穴，冰穴里面有一个小男孩和他的妹妹。他们被困在里面无法脱身。于是，他们便一起出发去寻找出路。历经了九死一生的探索，他们来到了一座神秘的冰山下。冰山下面是一处湖泊。



悟道1.0

- 悟道 • 文源: a Chinese oriented pre training language model
 - With 2.6 billion parameters, it has 20 mainstream Chinese natural language processing tasks, and its technical capacity has been level with gpt-3.
- 悟道 • 文溯: a pre-training model for large-scale protein sequence prediction
 - A super large-scale pre training model with one billion parameters and can deal with super long protein sequences under the guidance of gene domain cognitive map.

悟道1.0

- 悟道·文汇：a new cognitive oriented large-scale pre-training model

旗袍是一种古典的风格，它不仅能够彰显出女性优雅气质，而且还具有很好的修饰身形。这款旗袍采用了经典的圆领设计，穿着舒适自在，同时又能够展现出女性柔美的颈部线条，让你更加的迷人。精致的绣花工艺，使得整件衣服看起来更加的精致，也更加的凸显出女性的魅力。

- With 11.3 billion parameters.
- Wenhui is close to breaking through the Turing test.

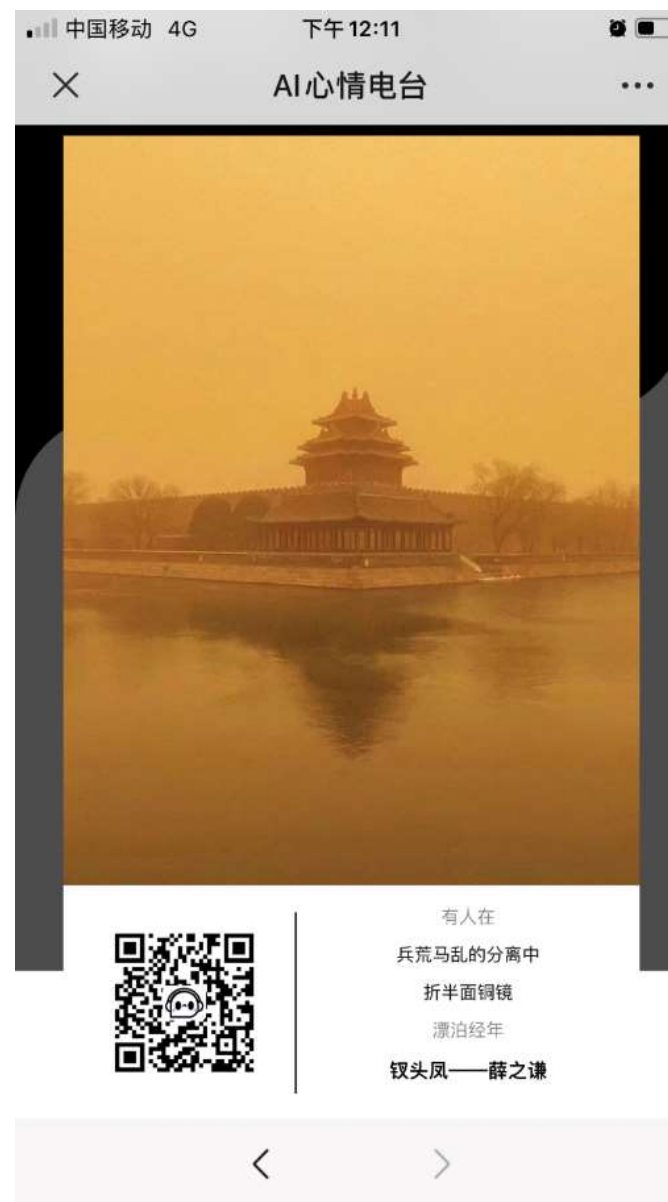


图：「悟道·文汇」模型基于图片自动撰写商品文案



悟道1.0

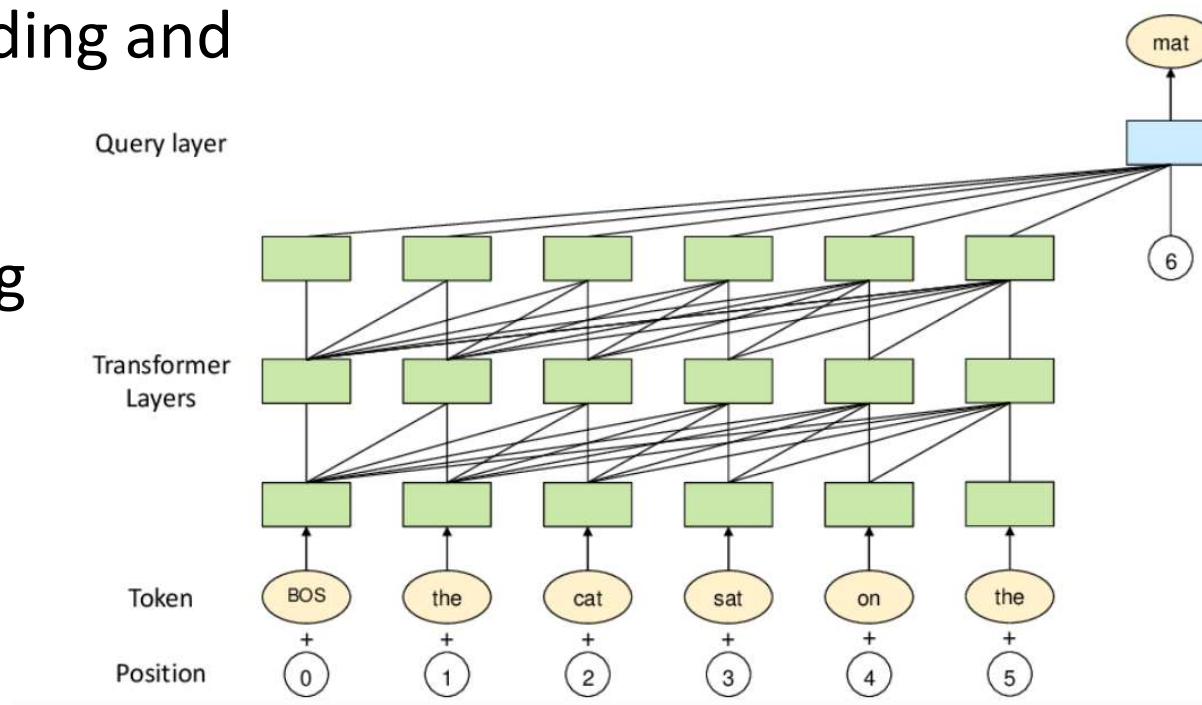
- 悟道 • 文澜: Chinese general graphic multimodal pre training model
 - break through the theoretical problem of pre training based on multimodal data combined with graphics, text and video
 - 1 billion parameters



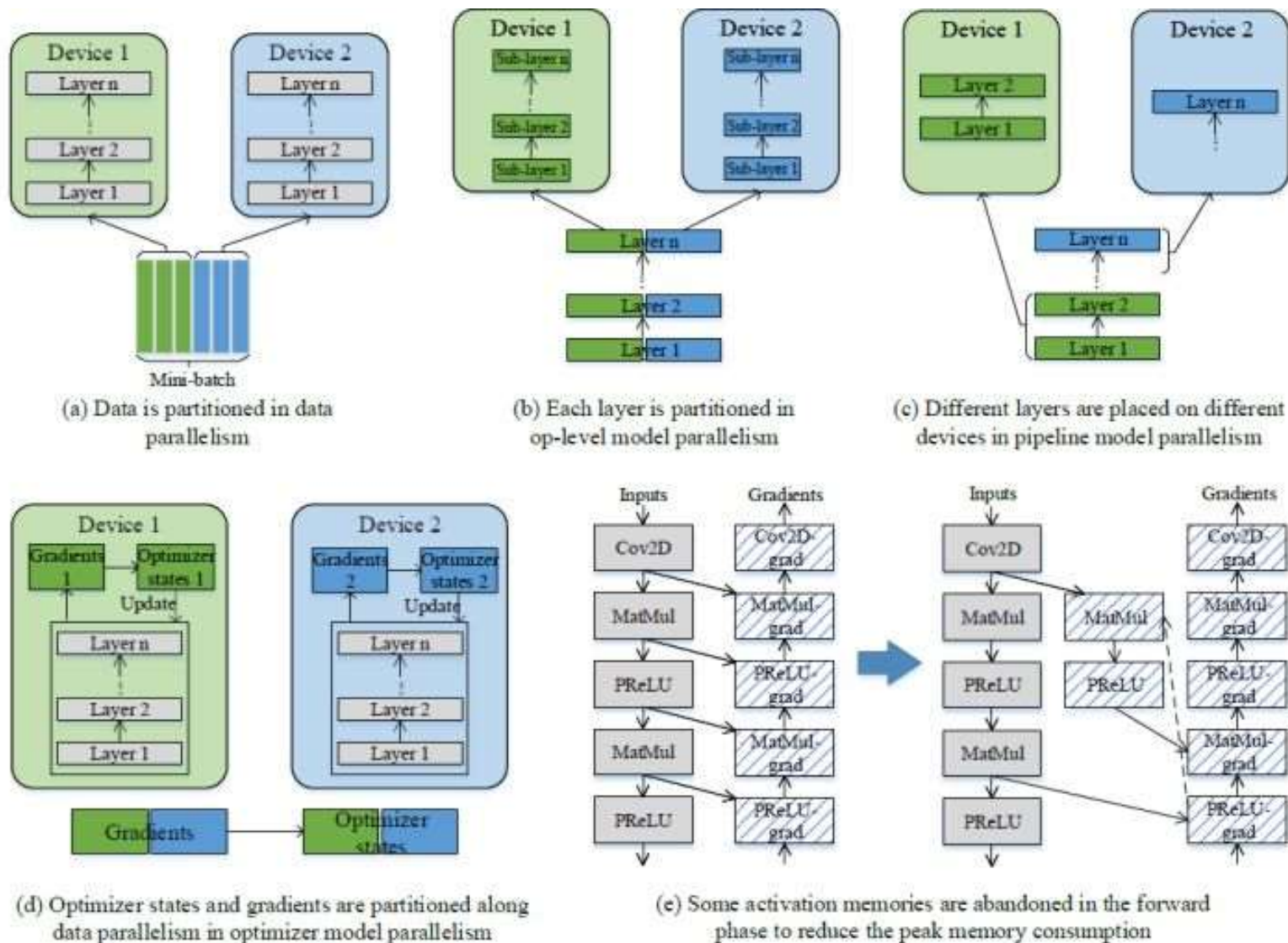


Huawei PanGu

- Random word order generation
- Predictor
- Support understanding and generating tasks
- Continuous learning



Huawei PanGu





Huawei PanGu

盘古
PanGu-Alpha

退出

鹏程·盘古- α (中文版)

业界首个2000亿参数以中文为核心的预训练生成语言模型。

注：本页面开放了盘古- α 130亿参数数量模型的测试版推理服务，长文本生成功能将于近期开放！
由于推理资源有限，若遇到排队状态请您耐心等待，感谢理解！

欢 | 迎 | 体 | 验

(可选) 增大topK或topP以提高生成结果多样性

topK: topP: 最大生成长度:

盘古 对对联

测试内容: 开始生成

队列位置:

上联: 瑞风播福泽, 事业昌盛千家乐

I

下联: 骏骥踏春来, 前程似锦万代兴

本项目处于前沿探索阶段，体验功能仅供学术测试使用。请勿输入违反法律法规内容，同时，未经许可，禁止分享、传播输入及生成文本内容。感谢理解！

盘古 对对联

盘古 问答

盘古 对话生成

盘古 续写小说

盘古 专业文档撰写

盘古 自由生成



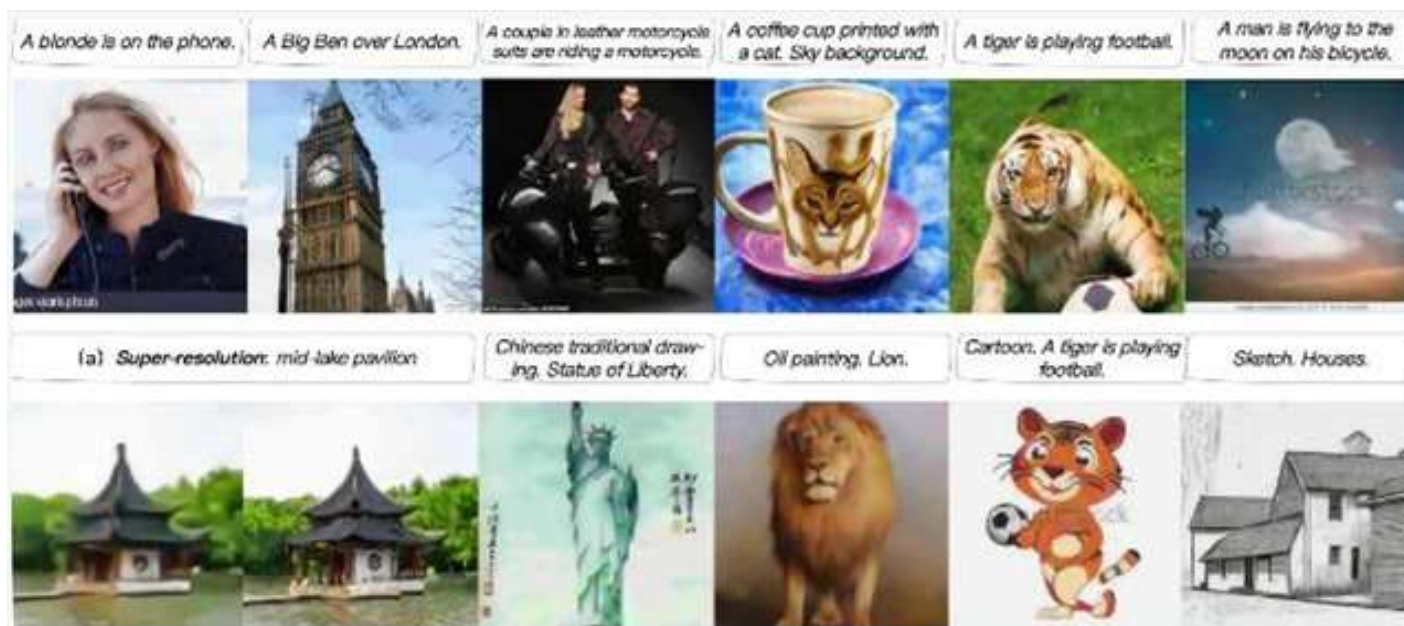
悟道2.0

- Artificial intelligence giant model
- With 1.75 trillion parameters
- The largest pre training model in the world.

“悟道2.0”超大规模预训练模型阵容	
名称	介绍
悟道·文汇	面向认知的超大规模新型预训练模型 成果1: 全球最大、中国首个万亿双语多模态模型 成果2: 最大的英文通用大规模预训练模型GLM2.0 成果3: 世界最大中文多模态生成模型CogView 成果4: 自回归模型在理解任务上首超自编码模型
悟道·文澜	超大规模多模态预训练模型 成果1: 文澜多模态多语言模型MLMM, 在图文检索、图像问答等任务均超过现有多模态多语言模型 成果2: 文澜BrVL双塔模型, 是首个中英文多模态预训练双塔模型、最大的中文多模态预训练数据、首个基于DeepSpeed的跨模态对比学习预训练算法, 在ImageNet和MSCOCO上超过CLIP和ALIGN
悟道·文源	以中文为核心的超大规模预训练语言模型 成果1: 兼顾理解与生成能力的中文预训练开源模型 成果2: 面向预训练模型的全流程高效计算框架 成果3: 中文机器语言能力权威评测平台 成果4: 世界首创法律领域长文本预训练模型
悟道·文溯	超大规模蛋白质序列预测预训练模型

悟道2.0

- Give a realistic scene or wild words, such as "blonde calling", "drawing a house", or "tiger playing football", "riding a bike on the moon alone",





悟道2.0



修身露脐小衫排扣半袖女针织T恤

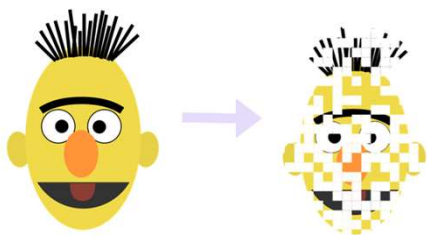


收口松垮收腿八分工装裤

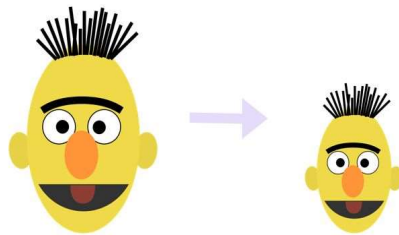


Model Compression

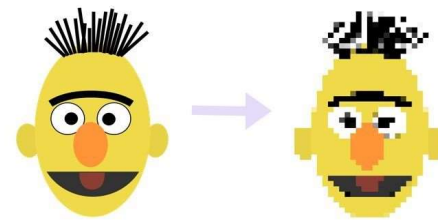
- Pruning: remove redundant parts in the model
- **Distillation**: use small models to mimic large models
- Quantification: Use low-bit representation, e.g. float32→int8
- etc.



Pruning



Distillation



Quantification



In-Context Learning

- Input: task description + examples + prompt

Few-shot

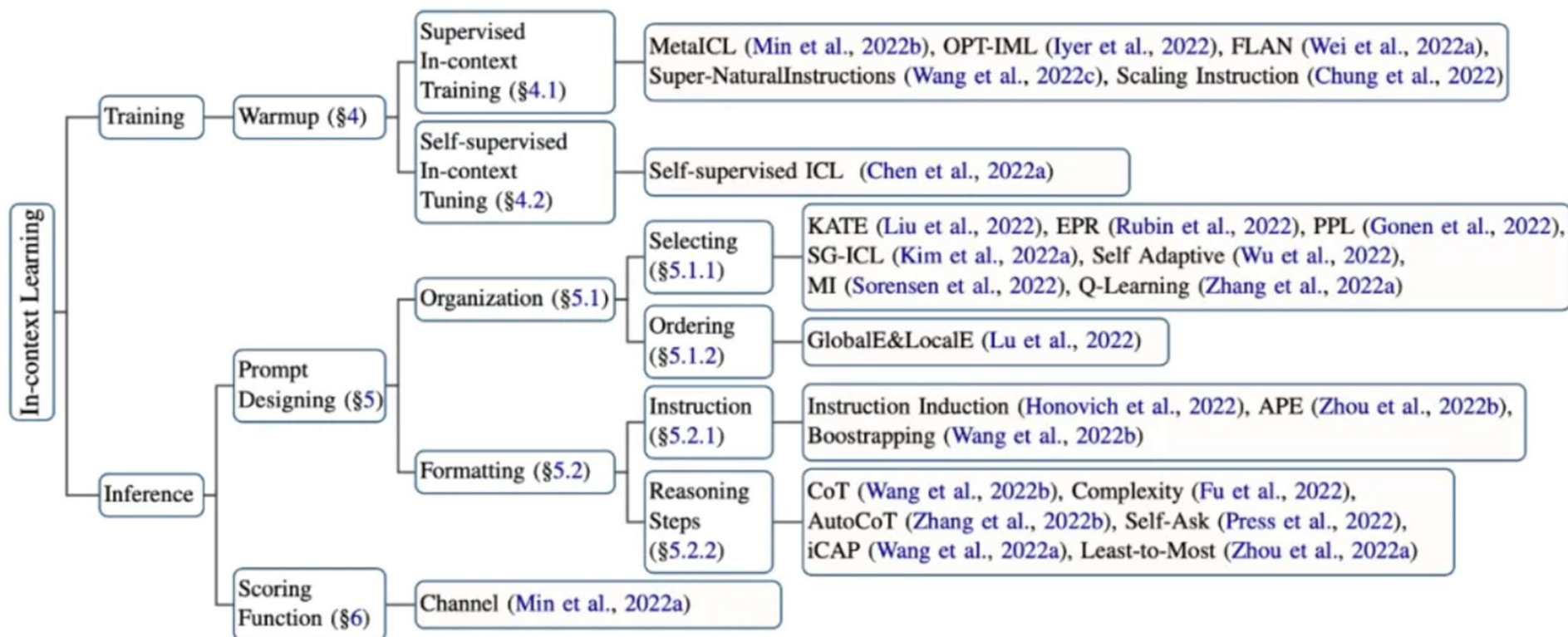
In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1  Translate English to French:      ← task description
2  sea otter => loutre de mer        ← examples
3  peppermint => menthe poivrée
4  plush girafe => girafe peluche
5  cheese => .....                ← prompt
```




In-Context Learning

- A Survey for In-context Learning





Reference

1. N-gram Language Models <https://web.stanford.edu/~jurafsky/slp3/3.pdf>
2. Recurrent Neural Networks and Language Models.
<http://web.stanford.edu/class/cs224n/slides/cs224n-2020-lecture06-rnnlm.pdf>
3. 神经网络语言模型
[http://www.nlpr.ia.ac.cn/cip/ZongReportandLecture/Lecture_on_NLP/NLP2018_Chapter_05_LM\(2-2\)_NLM.pdf](http://www.nlpr.ia.ac.cn/cip/ZongReportandLecture/Lecture_on_NLP/NLP2018_Chapter_05_LM(2-2)_NLM.pdf)
4. Contextual Word Representations with BERT and Other Pre-trained Language Models
https://web.stanford.edu/class/cs224n/slides/Jacob_Devlin_BERT.pdf



The next lecture

- Lecture 6: Word Processing